World Models for Coding: A Critical Survey of Execution-Grounded Code Models, Agentic Evaluation, and Overloaded Terminology

Keon Kim

May 2026

Contents

World Models for Coding: A Critical Survey of Execution-Grounded Code Models, Agentic Evaluation, and Overloaded Terminology

Last updated 2026-05

Genre note. This document is a hybrid: §§6–16 are a conventional survey of the field (foundations, lineages, benchmarks, empirical landscape), while §17 is an explicitly argumentative critical synthesis. Readers who want only the catalog can skip §17; readers who want only the criticism can read the Theses Summary below plus §17 and skip §§6–16. The two halves are meant to be read together — neither alone captures the field's current shape.

Abstract

A *world model* is an internal predictor an agent maintains over the dynamics of its environment, used to imagine the consequences of actions. In coding, the environment is the program itself — its runtime state, its execution trace, the filesystem it manipulates, the tests it must satisfy, the developer task it is solving. Twelve years after Zaremba & Sutskever asked whether a network could execute code (1410.4615), and seven months after Meta FAIR released CWM (2510.02387) as the first openly-released LLM explicitly branded a Code World Model, the question has changed. It is no longer controversial that internal models of execution are *learnable*; whether they are *necessary*, *architecturally separable*, or *causally responsible* for coding-agent gains remains unsettled. The open questions are whether the *named artifact* "code world model" is a structural commitment or a marketing label, whether trace pretraining buys what it claims, and whether the field's Dreamer-shaped vocabulary will survive when the empirical evidence comes in.

This survey synthesizes 184 papers around the world-model lens. We define the object of study with two distinct definitions — a permissive *descriptive* one (what the field currently calls a code WM) and a strict *normative* one (architectural commitment to forward state prediction); trace its twelve-year arc; build a three-axis taxonomy (functionality × temporal × representation); produce technical system cards for thirteen representative systems; assemble protocol-stratified benchmark tables for SWE-bench, CRUX-Eval, web agents, and formal verification; develop seven critical theses where the field overclaims; and identify the open problems where new work would matter most. The single most defensible empirical claim is that *execution-grounded supervision improves code agents on runtime-reasoning-heavy benchmarks*; the single most defensible critical claim is that broader inferences from this to "world models in coding" remain underdetermined by the evidence at this date.

Theses Summary (Read First)

The survey is a hybrid: catalog plus critique. Readers pressed for time can read this summary and §17 (Critical Perspectives) and skip the lineage chapters (§§6–14). Readers entering the field should read §§1–5 first, then §§6–16, then §17. The catalog and the critique are meant to be read together.

The seven critical theses developed in §17, each grounded in a specific disconfirmation from the corpus:

1. The "world model" label has become overloaded. Under the field's permissive descriptive definition (D, §3.1), CWM, TRACED, SemCoder, LLM-JEPA, and DyMo are all "world models" despite being architecturally standard LLMs with enriched training. A strict normative definition (N1/N2/N3, §3.1) admits only systems with explicit forward-prediction machinery: neural dynamics models (N1, no code exemplar), latent-action rollout models such as CoLA (N2), and synthesized executable simulators such as GIF-MCTS and ARC executable WMs (N3).

2. Trace pretraining has a causal-isolation problem. "Do Code Semantics Help?" (2509.11686) finds no trace representation consistently improves code synthesis across multiple backbones; CWM's 65.8% SWE-bench Verified is confounded between trace mid-training, ForagerAgent trajectories, and RL.

3. The Dreamer-for-code gap may be a non-problem. Vision needed latent rollouts because pixels are expensive; Python state is small and CPython is free, so the architectural pressure does not transfer. CWM in token space reaches state-of-the-art at 32B.

4. PRMs are critics, not world models. A learned evaluator of partial trajectories cannot roll out future states; the conceptual conflation dilutes the world-model vocabulary.

5. "General Agents Contain World Models" (2506.01622) is weaker than its title. The theorem applies under restrictive assumptions (full observability, stationarity, deep-composite-goal regret bounds) that SWE agents demonstrably violate.

6. The verifier-grounded lineage is the actual leading edge, but scoped. ATLAS, Re:Form, CLEVER, VeriStruct, AutoRocq produce machine-checkable correctness. Scoped: they lead on synthesis-from-spec (e.g., ATLAS DafnySynthesis 65.8% pass@5); they do *not* lead on end-to-end verified codegen from NL (CLEVER ?1/161 Lean).

7. The evaluation gap is the structural reason the field looks confused. No benchmark holds policy fixed and varies WM quality. Models with 85–98% output-prediction accuracy still produce traces with systematic errors throughout — outcome accuracy decouples from process fidelity.

These theses are the survey's main contribution. §§6–16 catalog the work; §§16–17 provide the empirical and critical synthesis that grounds the theses; §18 lists open problems.

---

1. Introduction

Autoregressive code LLMs generate tokens conditioned on syntactic context. Correct programs live in two worlds: a *syntactic* world of tokens and a *semantic* world of values, control flow, side effects, and developer intent. The world-model framing — imported from model-based reinforcement learning, where it names an internal predictor of environment dynamics used to imagine action outcomes — is the field's bet that the gap between these two worlds closes when the network has been trained on what code does rather than only on what code looks like.

Two adjacent surveys cover non-overlapping ground. A Survey on LLMs for Code Generation (2406.00515) maps the code-LLM space without the world-model lens. Understanding World or Predicting Future (2411.14499) maps world models in general without the code lens. A Comprehensive Survey on World Models for Embodied AI (2510.16732) maps embodied world models but excludes the coding domain. The intersection — the subject of this document — has cohered only recently into a recognizable program.

We aim for three things at once: comprehensive coverage of the corpus, technical depth on the canonical systems, and an honest accounting of where the prevailing rhetoric outruns the evidence. The first two are owed to readers entering the field; the third is owed to those already in it.

---

2. Methodology

Corpus construction. The 184-PDF corpus enumerated in `papers.json` was assembled in four iterative passes between March and May 2026. The seed paper was CWM (2510.02387). Each pass expanded the corpus along a different axis: (i) snowball search on cited and citing papers via Semantic Scholar's reference API and arxiv listings; (ii) targeted topic searches on subdomains where the previous pass was thin (latent-action WMs for LLMs; safety/malicious-code; symbolic-execution and formal verification; agent memory; REPL-grounded models); (iii) a 2026-specific sweep to capture papers the earlier passes missed because of arxiv-ID ambiguity; (iv) a focused fourth pass on reasoning models, process reward models, test-generation, decompilation, diffusion code models, mech-interp, ARC synthesis, self-improvement, Verilog/RTL, and major 2026 capstones. Each pass produced a written rationale for accepted and rejected candidates.

Inclusion criteria. A paper was included if it credibly intersected *both* (a) world-modeling, state-tracking, or environment-prediction architectures, and (b) code generation, debugging, repair, or agentic coding. Pure code-LLM papers without a world-model angle and pure world-model papers without a code angle were excluded. Where the boundary was ambiguous, we erred on the side of inclusion when the paper introduced a representation, training objective, or evaluation that could plausibly be adopted by code-WM work.

Exclusion criteria. Excluded: pure transformer scaling papers; pure vision world models (DreamerV1/V2/V3, V-JEPA, Genie) except where cited as architectural precedent; generic RL theory; classical PL foundations; benchmark surveys without methodological contribution. We retained DreamerV1–V3 as named citations in §11 because they ground the latent-action discussion, but did not include their PDFs in the corpus.

Date cutoff. Papers with arxiv submission dates through 2026-05-15. Arxiv-ID format YYMM.NNNNN means IDs beginning 2601, 2602, 2603, 2604, 2605 are valid 2026 papers (Jan–May 2026), not future-dated. An earlier survey pass mistakenly excluded these; the fourth pass corrected the omission.

Selection bias and reproducibility. Two biases are explicit. First, anglocentric arxiv-first selection — we did not systematically cover Chinese-language preprint servers, workshop proceedings without arxiv copies, or industry technical reports. Second, recency bias — 60% of the corpus is from 2025 onward, reflecting the field's actual growth curve but also our search recency. The full corpus is enumerated in `papers.json` with arxiv IDs, titles, and filenames; readers can regenerate the bibliography from this index.

Sources reconciled with adjacent surveys. We compared our taxonomy and corpus against three awesome-list repositories (knightnemo/Awesome-World-Models, JiahuaDong/Awesome-World-Models, tsinghua-fib-lab/World-Model) and two prior surveys (Ding et al. 2411.14499 on world models in general; Li et al. 2510.16732 on world models for embodied AI). The three-axis taxonomy in §5 is adapted from Li et al. with code-specific representation classes added. Where our judgment diverges from these adjacent works — for example, in our classification of CWM as architecturally a token-policy rather than a world-model proper (§17.1) — we mark the divergence explicitly.

Taxonomy coding. Each paper was assigned at least one lineage label (§§6–13) and at least one representation label (§5.3 table). The assignments were made by one of us and are recorded in the per-section enumerations and `README.md`. We have not yet performed inter-rater reliability checks; readers should treat the assignments as one curator's best judgment, not consensus.

Search procedure (for reproducibility). Seed: arxiv 2510.02387. Search engines: arxiv listing API (cs.SE,

cs.PL, cs.LG, cs.CL submission categories), Semantic Scholar Graph API (`/paper/arXiv:{id}/references` and `/paper/arXiv:{id}/citations`, fields=title,externalIds,abstract,year), and WebSearch with query variants enumerated below. Date cutoff: arxiv submissions through 2026-05-15.

Representative search queries used across the four passes (non-exhaustive): - "world model code generation", "code world model" - "neural code execution", "learning to execute" - "execution trace pretraining LLM", "trace-aware code model" - "Dreamer code", "JEPA LLM", "latent action LLM" - "SWE agent reinforcement learning", "agentic coding world model" - "self-debugging execution simulation", "trace-driven program repair" - "symbolic execution LLM", "verified code generation Dafny", "Lean code agent" - "diffusion code model", "decompilation LLM semantic equivalence" - "Verilog RL world model", "ARC executable world model" - "process reward model code", "long-CoT code reasoning"

Inclusion / exclusion counts. Pass 1 considered ~70 candidates, accepted 31. Pass 2 considered ~40, accepted 20 (focused on 2026 papers earlier passes missed). Pass 3 considered ~80, accepted 52 via citation BFS from 11 central papers. Pass 4 considered ~60, accepted 28 across 5 thin niches. Final corpus: 184 PDFs (one was added late: 2510.16732). Total candidates considered: ~250. Rejection reasons recorded per-paper in the pass reports.

Evidence grading. Each citation in this survey can be classified by source type. We do not annotate inline (it would clutter the prose), but the following hierarchy is the implicit standard:

Tier	Definition	Survey use
A	Peer-reviewed venue (NeurIPS, ICML, ICLR, ACL, OOPSLA, FSE, etc.)	Highest weight; load-bearing claims
B	Major arxiv preprint with code/checkpoints + multi-author institutional backing	Treated as A for technical accuracy, less for community vetting
C	Solo-author or single-institution arxiv preprint	Cited for ideas; numerical claims hedged
D	Technical report / blog / benchmark leaderboard claim	Cited as evidence of capability, not as scientific finding

Roughly 60% of the corpus is currently Tier B; ~25% Tier A (mostly pre-2024); ~12% Tier C; ~3% Tier D. The recency skew toward 2025–2026 means many headline claims are Tier B and have not yet been independently replicated. Numbers in §16 should be read accordingly — they are the best available evidence at this date, not settled science.

Limitations of this methodology. Four. (1) No formal inter-rater reliability metric — one curator did the taxonomy coding. (2) No systematic verification of the headline numerical claims in every cited paper; §16 numbers were verified by reading the source PDFs but §§7–14 inline claims rely on abstracts and our prior reading. (3) The corpus is unstable — between drafts several 2026 papers we initially excluded as off-topic turned out to be relevant. A fifth pass would change the count by ±15 papers without materially changing the survey's conclusions. (4) Selection bias toward arxiv-first, anglophone, code-LLM-adjacent sources; we did not systematically cover Chinese-language preprint servers, workshop proceedings without arxiv copies, or industry technical reports.

3. Defining a World Model for Coding

The literature uses "world model" in two distinct senses that this survey will carefully separate.

3.1 Two definitions

Definition D (descriptive, permissive). A *world model for coding* is any code LLM whose training objective concretely encodes program semantics — execution traces, runtime state, environment feedback, or simulated outcomes — in addition to or instead of source-token prediction. This is the field's current usage. Under D, CWM, TRACED, SemCoder, LLM-JEPA, DyMo, RLEF, and most papers in the corpus are world models.

Definition N (normative, strict). A *world model for coding* is a system with an explicit, separable forward-prediction mechanism. We split N into three sub-types:

- N1 — Neural dynamics model. A learned `W : (state, action) → next_state` instantiated as a distinct architectural component with latent recurrent state, separate dynamics head, or inverse model. Dreamer-class. No code exemplar in the corpus.
- N2 — Latent-action model. A learned action abstraction over a base LLM, with the LLM serving as transition model in compressed action space, supporting rollout or tree search over latent actions. CoLA (2503.21383) is the canonical example.
- N3 — Synthesized executable simulator. The world model is an executable program (Python, DSL) synthesized by an LLM and run against ground-truth transitions during planning. GIF-MCTS (2405.15383), WorldCoder, Executable WMs for ARC-AGI-3 (2605.05138).

Each N-subtype has a distinct architectural commitment. N1 commits to learned latent dynamics; N2 commits to a discrete latent-action space; N3 commits to program synthesis as the dynamics. Conflating them — as the loose "Dreamer-for-LLMs gesture" framing did — obscures which architectural bet is being made. Under any N-subtype, CWM, TRACED, SemCoder, and most of the corpus do *not* qualify.

The two definitions agree on the empirical fact that execution-grounded supervision helps. They disagree on whether that grounding is correctly named a *world model* in the model-based-RL sense.

Aspect	Definition D (descriptive)	Definition N (normative)
Granted to	Any LLM trained with execution-related signal	Only systems with explicit forward-prediction module
Includes CWM	Yes	No (architecturally a Llama-class decoder)
Includes TRACED, SemCoder, NExT	Yes	No (auxiliary heads on a Transformer)
Includes CoLA, GIF-MCTS	Yes	Yes
Includes PRMs (ExecVerify, ThinkPRM)	Often grouped	No (critics, not forward predictors)
Includes verifiers (Lean, Dafny, Z3)	Sometimes grouped	No (deterministic oracles, not learned WMs)
Includes Dreamer / V-JEPA (vision precedents)	Yes	Yes

This survey uses D throughout the catalog sections (§§6–16) because that is how the literature talks,

and N in §17 because the critical synthesis depends on it. We mark the switch each time it matters. The split has been called for in prior reviews of the field and the two-definition framework is, in our reading, the cleanest resolution.

3.2 What is modeled

Orthogonal to D-vs-N, a fourth question asks *what* is modeled. The corpus splits across:

- variable values and stack frames (CWM, CodeExecutor)
- linear or branching execution traces (NExT, SemCoder, TRACED)
- test outcomes and runtime errors (LEVER, RLEF)
- environment/OS/web state (WebDreamer, Dyna-Think)
- repository state (RepoGraph, Understanding by Reconstruction)
- developer task or specification (ATLAS, Re:Form)
- adversarial behavior (Double Life of CWMs)

A given system typically commits strongly to one or two of these.

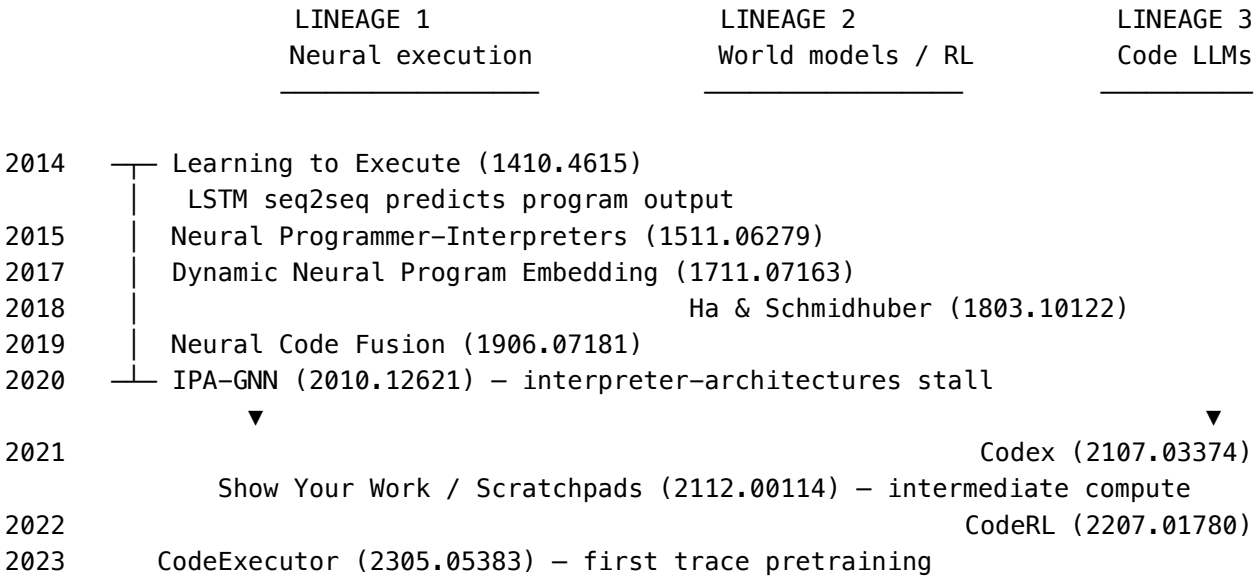
3.3 Behavioral vs architectural classification

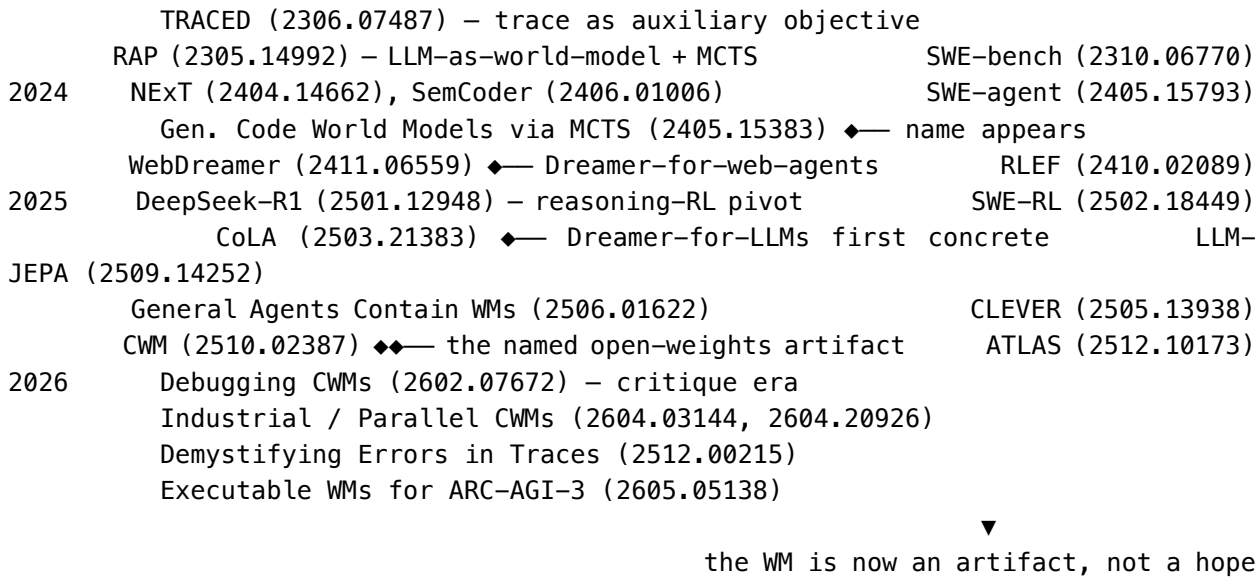
CWM occupies an instructive position. *Behaviorally*, it emits stack frames and variable bindings, which feels like an "explicit symbolic world model." *Architecturally*, it is a 32B Llama-class decoder with no separate dynamics head, RSSM, or inverse model. The two readings are both correct: CWM is behaviorally explicit but architecturally implicit. Earlier framings in the literature (and earlier drafts of this survey) collapsed these into a single "explicit WM" label; we recommend separating them. SemCoder, NExT, and TRACED follow the same pattern.

---

4. Twelve Years of Code World Models

The lineage is best understood as a sequence of inheriting questions. Each era's answer dissolved the previous era's bottleneck and exposed the next.





Diamonds (?) mark moments where "world model" enters the *name* of the contribution.

Pre-2020 — Can a network execute code? Zaremba & Sutskever's Learning to Execute (1410.4615) handed an LSTM character-level Python and asked it to predict output. The model worked on straight-line programs with bounded loops, but only with a curated curriculum, and only because the LSTM's constant memory was just enough to simulate the interpreter when the interpreter ran in constant memory too. Everything that followed in this lineage tried to escape that curse. Neural Programmer-Interpreters (1511.06279) and the Differentiable Forth Interpreter built differentiable program counters and call stacks — the bet that the right architecture would close the gap. Dynamic Neural Program Embedding (1711.07163) made the inverse move: run the real interpreter, embed the resulting state traces. Neural Code Fusion (1906.07181) and IPA-GNN (2010.12621) extended the GNN-over-execution playbook to the point where attention played the role of the program counter. By 2020 the lineage had answered its question — yes, a neural network can play interpreter, but only when the interpreter is encoded into its architecture, and these architectures did not transfer to Python, C, or assembly at corpus scale. Ha & Schmidhuber's World Models paper (1803.10122) had already named for vision and RL exactly the pattern this lineage was reaching for. The vocabulary existed; the coding community had not yet borrowed it.

2020–2022 — Can the network's training include execution? The era's reframing was simple: instead of *can the network execute*, ask *can we train a normally-shaped Transformer on enough execution evidence that semantics seep into its weights.* Codex (2107.03374) and MBPP (2108.07732) made code generation a real engineering target. Show Your Work / Scratchpads (2112.00114) made the decisive move: a Transformer that could not predict a program's output could predict it perfectly if it was allowed to emit the intermediate computation first. Same trick Dynamic Neural Program Embedding had pulled, now at LLM scale, in token space, without architectural surgery. The neural-as-interpreter program of 2014–2020 did not scale beyond toy languages; the next era's move was to bake execution into training data rather than architecture.

2023 — Trace pretraining as a named recipe. CodeExecutor (2305.05383) made the recipe explicit: mutate competitive-programming submissions, run them in a sandbox, capture per-line state tokens, train a transformer to emit the trace from source. TRACED (2306.07487) generalized this to a pre-training auxiliary that any code-LLM could absorb. CRUXEval (2401.03065) provided the canonical input/output-prediction benchmark and suddenly there was a number that captured "does this model

understand what code does, as opposed to what code looks like." In parallel, Reflexion (2303.11366) and Self-Debug (2304.05128) showed that an LLM's mistakes could be fed back to itself as natural-language critiques, LEVER (2302.08468) used execution to verify candidate generations during decoding, and RAP (2305.14992) framed the LLM itself as a world model and ran MCTS over its imagined rollouts. The year's unifying insight: execution traces are an auxiliary objective, not a separate model.

2024 — From models that simulate to agents that act. SWE-bench (2310.06770) replaced "write a function that passes a unit test" with "fix a real GitHub issue in a real repository." CodeAct (2402.01030) claimed the agent's action space should be Python code itself. SWE-agent (2405.15793) shipped the harness. NExT (2404.14662) inlined traces into the agent loop. RLEF (2410.02089) fed execution outcomes back as RL rewards. WebDreamer (2411.06559) transplanted Dreamer-style imagination to digital agents. Generating Code World Models via MCTS (2405.15383) introduced the literal phrase "Code World Models." What unified the year was a structural shift in *where* the world model lives: in 2023 it lived in the weights, surfaced through trace prediction; in 2024 it lived in the loop, in the agent's behavior under environmental feedback.

2025 — The CWM moment. DeepSeek-R1 (2501.12948) opened the year by showing that pure reasoning-RL on verifiable rewards could reach the frontier on math and code. SWE-RL (2502.18449) applied the same recipe to full SWE traces. CoLA (2503.21383) made the first concrete attempt at a Dreamer-for-LLMs: inverse-dynamics over latent actions, then RL over a learned codebook. LLM-JEPA (2509.14252) ported LeCun's joint-embedding predictive objective to language. General Agents Contain World Models (2506.01622) supplied a theorem: any agent satisfying a regret bound on goal-conditioned tasks must have learned a predictive model of its environment. Then in October, Meta FAIR released CWM (2510.02387) — a 32B open-weights model mid-trained on 5T tokens of Python execution traces and ForagerAgent trajectories from Dockerized repositories. The thing the 2014 LSTM was trying to be was now a downloadable checkpoint.

2026 — Stress-testing and broadening. With the artifact in hand, the field pivoted to critique and generalization. Debugging Code World Models (2602.07672) catalogs CWM's failures on long traces and string-state representation. Demystifying Errors in LLM Reasoning Traces (2512.00215) audits where trace-trained models hallucinate. Industrial CWM (2604.03144) and Parallel-Code WMs (2604.20926) generalize the recipe to Verilog/GPU and parallelism semantics. Executable World Models for ARC-AGI-3 (2605.05138) brings the generative-environment flavor to abstract visual reasoning. Reinforcement World Model Learning for LLM Agents (2602.05842) flips the standard recipe by training the WM rather than the policy.

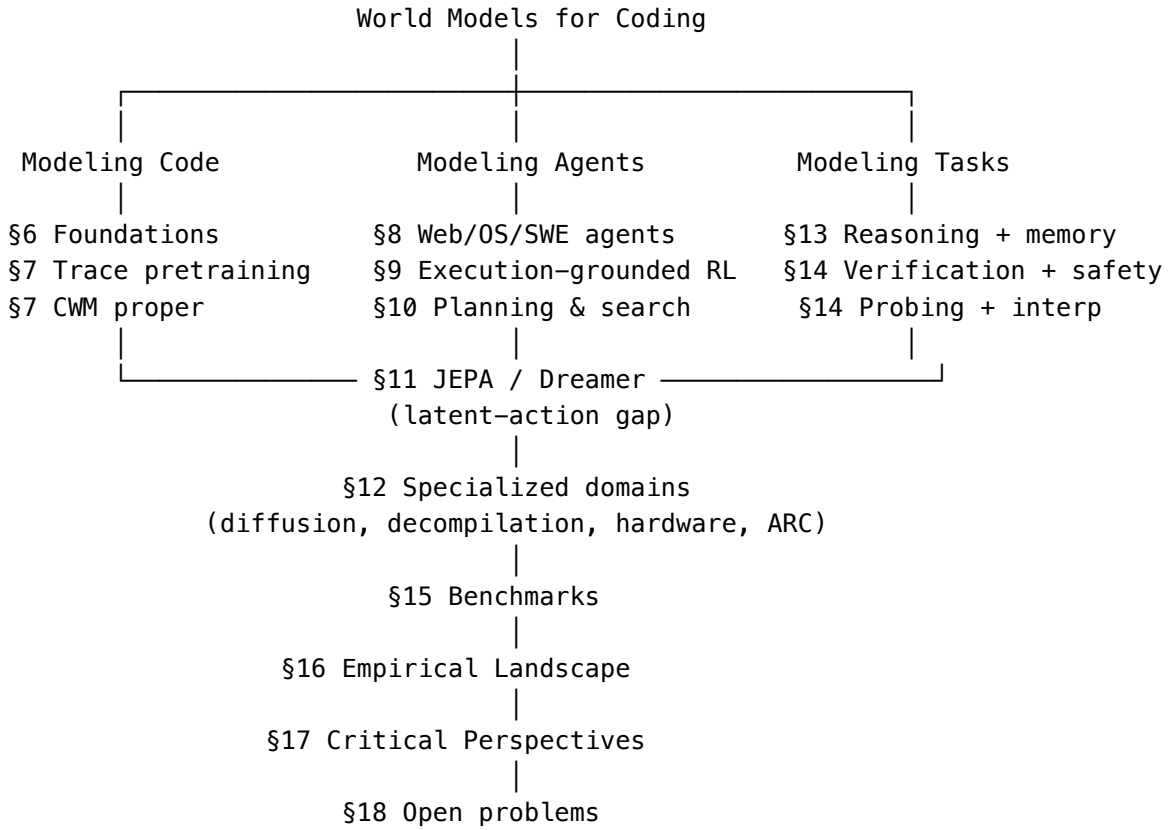
The arc. Across twelve years: *can a network execute code?* → *can a network's training include execution?* → *can an LLM agent simulate its environment?* → *is the world model a named artifact rather than a metaphor?* Each era's answer dissolved the previous bottleneck and exposed the next. Architecture gave way to data; data gave way to agency; agency gave way to artifacts. What was a half-philosophical question in 2014 — *does this network understand what code does* — became, in 2025, an operational one with an open-weights baseline. The field has not finished. The 2026 critique wave shows the artifact is brittle in ways the 2014 LSTM was never asked to be. But the trajectory is now legible.

---

5. Taxonomy: Three Axes of Code World Models

Two cuts at the taxonomy are useful, and they are complementary. The first is a *lineage* cut — which research thread produced the system — and is the basis for §§6–13. The second, more durable cut

adapts the three-axis framework of Li et al. (2510.16732, *A Comprehensive Survey on World Models for Embodied AI*) to the code domain. The lineage map is below; the three axes are developed in §§5.1–5.3.



Adjacent WM surveys converge on overlapping splits that the three-axis framework subsumes as projections. Ding et al. (2411.14499) split top-level by *implicit representation* vs *future prediction*. JiahuaDong's awesome-list organizes by *paradigm*: RL-based / observation-generative / latent-space / object-centric. knightnemo's list surfaces *pixel vs mesh vs latent* as cross-cutting tags. The three axes — functionality, temporal modeling, and representation — capture the choices a system makes regardless of which lineage it belongs to.

5.1 Axis 1 — Functionality

- Decision-coupled WMs model only the slice of the world relevant to acting on it. CWM (2510.02387), RLEF (2410.02089), and WebDreamer (2411.06559) are decision-coupled — their WMs exist to enable code generation, RL planning, or web navigation respectively. CWM does not predict global filesystem state; it predicts the next Python frame *because* the next action depends on it.
- General-purpose WMs model the environment without reference to a particular task. The "general agents contain world models" theorem (2506.01622) is the abstract limit. In the corpus, only the largest CWM-class models with broad mid-training approach generality; most "code world models" are decision-coupled to a sub-task (repair, completion, agent control).

5.2 Axis 2 — Temporal Modeling

- Sequential simulation/inference. Step-by-step autoregressive rollout. CWM, NExT, SemCoder, all the trace-pretraining systems, and most LLM-as-WM planners (RAP, WebDreamer in its MPC loop) live here. The state is updated one timestep at a time. The vision analog is RSSM (Hafner et al., DreamerV1–V3).
- Global difference prediction. Predict the entire future state at once, in parallel. The vision analog is video-diffusion or masked-JEPA. In code, this fits diffusion code models (DiffuCoder, 2506.20639; Dream-Coder 7B, 2509.01142) where the next state is sampled jointly rather than autoregressively, and the "specification is the program" framing (2603.17399) where the entire trace is the spec.
- Static, no-trace. Some systems (SemCoder's static mode, the trace-free baselines in "Do Code Semantics Help?") explicitly drop temporal modeling at inference, reducing to single-shot prediction.

5.3 Axis 3 — Representation

This is the axis where the WM literature has converged most strongly, and where the code-WM literature is most uneven. Adapting Li et al.'s four-category split (GLV / TFS / SLG / DRR) to code yields five classes, of which only two are well-populated.

Class	Encodes the world as	Vision analog	Code exemplars
Token Sequence (TS)	Discrete or continuous token streams with execution traces, variable bindings, or rationales interleaved with source	Token-as-pixel (IRIS, TWM, Genie, Sora)	CWM (2510.02387), CodeExecutor (2305.05383), TRACED (2306.07487), NExT (2404.14662), SemCoder (2406.01006) — the dominant code-WM mode
Global Latent Vector (GLV)	A compact vector updated recurrently, encoding the entire program/agent state	RSSM (Hafner et al., DreamerV1–V3)	No clean exemplar. CoLA (2503.21383) introduces a learned action codebook but is otherwise a standard LLM, not RSSM-style
Spatial / Structural Grid (SLG)	A geometric or structural grid (BEV/voxel in vision; AST, call-graph, CFG in code)	OccWorld, DriveWorld	No exemplar. RepoGraph (2410.14684) uses a static dependency graph but does not predict over it as a WM
Decomposed Object / Slot (DOR)	Distinct persistent latent slots for objects in the world	SlotFormer and object-centric WMs	No exemplar. No code-WM models variables, scopes, or classes as discrete persistent slots

Class	Encodes the world as	Vision analog	Code exemplars
Synthesized executable (N3)	The world model *is* an executable program, synthesized rather than learned	(orthogonal to vision)	GIF-MCTS (2405.15383), WorldCoder, Executable WMs for ARC-AGI-3 (2605.05138)

Verifiers (Lean, Dafny, Z3) and PRMs are intentionally absent from this table. A verifier is not a *representation* of the world; it is a *grounding oracle* over candidate artifacts. A PRM is a *backward-looking critic*, not a forward predictor. Both belong in §5.4 below.

Three white spaces. The Dreamer-style GLV, the object-centric DOR, and the spatial-grid SLG representations have not been instantiated for code, despite being mature for vision. §18 lists these as open problems, with the caveat that not all three are equally promising — the Dreamer-style gap is contestable (§17.3), while the object-centric and structural-grid gaps look more genuine.

5.4 Grounding mode (orthogonal to representation)

A second classification asks how each system's predictions are validated. This is the analog of Li et al.'s "reality" column and addresses the decoupling between WM-head accuracy and downstream task success that DyMo (2506.02918) and §17.7 develop.

Grounding mode	Definition	Code exemplars
None / self-report	Predictions never validated against ground truth	RAP (2305.14992), basic LLM-as-WM planners
Execution-grounded	Predictions checked against real interpreter / runtime	CWM (2510.02387), TRACED (2306.07487), NExT (2404.14662), SemCoder (2406.01006), RLEF (2410.02089)
Verifier-grounded	Outputs checked by Lean / Dafny / Verus / Rocq / Z3	ATLAS (2512.10173), Re:Form (2507.16331), CLEVER (2505.13938), VeriStruct (2510.25015), AutoRocq (2511.17330)
Synthesized-simulator-checked	Synthesized world model checked against held-out transitions	GIF-MCTS (2405.15383), Executable WMs for ARC-AGI-3 (2605.05138)
Critic-grounded (not a WM)	Backward-looking value/quality score, no rollout	ExecVerify (2603.11226), SWE-PRM (2509.02360), ThinkPRM (2504.16828) — listed for contrast; these are critics, not WMs (§17.4)

Grounding mode is orthogonal to the representation axis: a token-sequence representation can be

execution-grounded (CWM) or self-report (RAP); an N3 synthesized-simulator can be checked against transitions (GIF-MCTS) or never validated. The "WM fidelity" question of §17.7 is, structurally, a question about whether a system's grounding mode is strong enough to falsify a bad WM at training time.

---

6. Foundations: Neural Execution as Implicit World Modeling

Zaremba & Sutskever's Learning to Execute (1410.4615) established both feasibility and brittleness. Show Your Work — Scratchpads (2112.00114) is the hinge moment: by training a Transformer to emit intermediate computation states, the authors recovered much of the LSTM-era execution-prediction performance at scale, presaging the trace-pretraining lineage of §6. CRUXEval (2401.03065) and REval (2403.16437) provide the canonical execution-reasoning benchmarks. The lesson the field absorbed: *replacing* the interpreter with a neural network is harder than *augmenting* a transformer with interpreter-style supervision. Modern systems all take the latter path.

---

7. The Trace-Pretraining and CWM Lineage

7.1 Trace-pretraining as a recipe

CodeExecutor (2305.05383) trains a Transformer to simulate Python execution token-by-token. TRACED (2306.07487) adds dynamic-state supervision to a code-LLM pretraining mix. NExT (2404.14662) formats traces as natural-language rationales, letting a chat-style LLM reason about runtime behavior via chain-of-thought. SemCoder (2406.01006) generalizes to "monologue reasoning" linking source-text to execution state.

The 2025 wave consolidated and stress-tested the approach. "What I cannot execute, I do not understand" (2503.05703) trains and evaluates LLMs explicitly on traces with dynamic scratchpads, pushing Llama-3.1-8B from 37.8% to ~80% on CRUXEval-O. Code Execution as Grounded Supervision (2506.10343) repurposes line-by-line traces as verifiable CoT. Self-Execution Simulation (2604.03253) lets the model train on its own execution predictions. Demystifying Errors in LLM Reasoning Traces (2512.00215) audits where trace-trained LLMs fail. "Do Code Semantics Help?" (2509.11686) is the most damaging paper in the lineage: a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five trace representations finds that *no single representation consistently improves code generation*, and in 7 of 9 synthesis settings the no-trace baseline wins or ties.

7.2 Deep dive — TRACED (2306.07487)

What it models	Per-line program state (concrete variable values quantized into 30 bins crossing data-type × value-range) and per-line execution coverage of a C program, statically predicted from source.
Data	CodeNet C subset: 1,805/1,900 problems, 121,319 training traces collected via `gdb` stepping through `-g -O0` builds.

Architecture / objective	RoBERTa-base initialized from UnixCoder. Three jointly-optimized heads on input `[CLS]` $e_1..e_i$ `[SEP][SEP]` $c_1..c_j$ `[SEP]`: MLM, per-variable program-state classification (`data_type, value_type, quantized_value`), per-variable binary coverage.
Headline results	Static execution full-path accuracy 71.6% vs UnixCoder 63.7% (+12.4% relative); variable-value accuracy 89.2% vs 87.8%; POJ-104 clone-retrieval MAP@R 91.2 vs 89.5.
Distinctive contribution	An early demonstration that *quantized* variable-value prediction is a viable pretraining signal — concrete values lose to discretized bins.

7.3 Deep dive — NExT (2404.14662)

What it models	Program execution as inline-comment trace appended to source: each statement annotated `# (k) varA=...; varB=...` capturing changed variables in execution order.
Data	Mbpp-R (10,047 train / 1,468 dev repair tasks built from incorrect LLM outputs on MBPP) and HumanEvalFix-Plus. Base model: PaLM 2-L.
Training objective	STaR-style self-training. Sample 32 (rationale, fix) candidates at T=0.8 from current model; filter by unit-test pass; SFT on accepted. Always restart from initial checkpoint each iteration. 10 iterations.
Headline results	Mbpp-R pass@1 23.2 → 49.3 (+26.1 abs); HumanEvalFix-Plus 32.2 → 42.5 (+10.3). Crucially, generalizes when traces are removed at test time (40.8 vs 23.2).
Distinctive contribution	Inline-comment trace format fits ~95% of MBPP into 2K window (vs ~60% for Scratchpad), and the trained model retains benefit even without traces at inference.

7.4 Deep dive — SemCoder (2406.01006)

What it models	Four jointly-trained semantics modalities: Approximate (NL docstring), Symbolic (source), Operational (execution effects), Abstract (input-invariant properties).
Data	PYX — synthetically curated, retried by generator until each sample executes; 4.3k decontaminated CodeContest problems for SemCoder-S. Base: DeepSeekCoder-6.7B-base.
Training objective	Standard NTP with loss on code + monologue tokens. Forward monologue verbalizes execution step-by-step (summarizing loop iterations rather than dumping every state). Backward monologue characterizes possible prior states given output (abstract constraints + concrete witness). Both rejection-sampled against ground-truth execution.
Headline results	HumanEval+ 79.3% / MBPP+ 79.9% / LCB-Lite 27.5% at 6.7B — beats GPT-3.5-turbo. CRUXEval-I 63.6 / CRUXEval-O 65.1 vs GPT-3.5-turbo 50.3 / 59.0. Monologue beats Scratchpad (48.8) and NExT (49.4) on CRUXEval-I: 61.8.
Distinctive contribution	Forward AND backward monologues (NExT is forward-only); abstract semantics constraints rather than concrete state at every step; entirely static at inference.

7.5 Deep dive — CWM (2510.02387)

What it models	Python interpreter state at the granularity of an *interpreter stack frame* (one observation–action pair per executed line), plus agentic SWE trajectories where actions are shell tool calls and observations are environment responses.
Architecture	32B dense decoder-only Transformer with GQA, sliding-window blocks, RoPE — Llama-class. No separate dynamics head, no inverse model, no recurrent latent.

Data composition	Four-stage train: (i) 8T-token general pretraining at 8k ctx; (ii) 5T-token code-world-modeling mid-training at 131k ctx: 120M traced Python functions, 262k CodeContests traces, ~70k repo-level traced commits, 75M natural-language trace rewrites, and 3M ForagerAgent trajectories from 10.2k Docker images / 3.15k repos (55% issue-fix, 45% synthetic mutate-fix); (iii) 100B-token SFT at 32k; (iv) 172B-token joint RL at 131k.
Trace format	Per-line `<|frame_sep|>{locals JSON}<|action_sep|>{source line}` so that next-token prediction *is* next-state prediction at line granularity.
Headline results	65.8% SWE-bench Verified with TTS (best@16 over 40 verifier-reranked samples); 68.6% LiveCodeBench-v5; 94.3% CRUXEval-Output; competitive with much larger closed models.
Distinctive contribution	To our knowledge the first open-weights model where line-level Python execution traces and large-scale (3M) agent–environment trajectories are mid-training data, not post-training data. Introduces *Activ* (using GitHub Actions CI for local image builds) to scale executable repository images.

Important caveat (developed further in §17): the 65.8% headline is *not* pure pass@1 but best-of-16 with verifier reranking. Pure pass@1 is approximately 53–55%. The trace-mid-training contribution is not causally isolated from the ForagerAgent-trajectory contribution and from the joint-RL contribution. Without an ablation removing one while holding the others fixed, the "world model" component's causal role is unfalsifiable.

7.6 Direct descendants of CWM

- Debugging Code World Models (2602.07672) — probes where CWM fails on long traces and string state; finds long-horizon failures are dominated by *action hallucination*, not state-propagation error.
- Learning Reasoning World Models for Parallel Code (2604.20926) — predicts race conditions and profiling artifacts from parallel source.
- Industrial CWM / InCoder-32B-Thinking (2604.03144) — CWM recipe on Verilog and GPU execution traces.
- The Double Life of Code World Models (2512.13821) — CWM trace predictions repurposed for malicious-behavior detection.
- Towards a Neural Debugger for Python (2603.09951) — neural debugger as forward/inverse world model.

- Neural Computers (2604.06425) — video-model-style WMs of CLI/GUI runtime from I/O traces.
- Generating Code World Models with LLMs Guided by MCTS (2405.15383) — the WM is *the code itself*, synthesized by an LLM.
- General Agents Contain World Models (2506.01622) — proves that sufficiently competent goal-conditioned agents must contain extractable world models, under restrictive conditions discussed critically in §17.

7.7 Deep dive — GIF-MCTS / Generating Code World Models via MCTS (2405.15383)

What it models	The world model itself is *Python code* — an `Environment.step(s,a) → (s', r, done)` class synthesized by an LLM to match a small set of pre-collected `(s, a, r, s', d)` transitions.
Data	CWMB benchmark — 18 RL environments (classic control, PyGame, MuJoCo) with NL descriptions and 5 random + 5 above-threshold demonstrations each. Plus APPS-Competition and RTFM.
Algorithm	Inference-time MCTS over partial programs with three action types: generate (append L=2 lines), improve (rewrite full program given failing transition), fix (repair runtime/syntax error). Reward = fraction of transitions correctly reproduced.
Headline results	APPS-Competition 28.3% pass@20 (Llama-3-70B), beating WorldCoder 25.1%. CWMB normalized return 0.76 vs WorldCoder 0.60. RTFM: GPT-4 reaches 1.00 accuracy.
Distinctive contribution	The world model *as code to be searched over*, not as a neural net to be trained. Once compiled and validated, runs 4–6 orders of magnitude faster than calling the LLM as WM.

8. World Models for Code Agents

Once an LLM is an *agent* taking actions in a non-trivial environment, the world-model question becomes whether the agent simulates the environment's response. Three sub-environments dominate.

8.1 Web agents

Web Agents with World Models (2410.13232) systematizes the thread. DyMo / World Modeling Improves LM Agents (2506.02918) adds a next-state prediction head to function-calling agents and reports gains on BFCL-V2 — though with a caveat (§17): the WM head reaches 90–94% state-prediction accu-

racy while the underlying policy reaches only 72.8% task success, illustrating that WM-head accuracy and agent accuracy can decouple.

Deep dive — WebDreamer (2411.06559)

What it models	Web environment as a POMDP where the LLM imagines natural-language state-change descriptions for each candidate action.
Data	3.1M synthesized web-interaction instances from Common Crawl URLs via biased random walking; state changes captured as before/after VLM screenshots described by Qwen2-VL-72B.
Training	Dreamer-7B trained from Qwen2-VL-7B on `(initial state, action) → state-change description`. Horizon H=1 empirically optimal.
Planning	Model Predictive Control — simulate each candidate action, GPT-4o scores trajectories on a 3-scale rubric, argmax executes.
Headline results	VisualWebArena 23.6 vs reactive 17.6 (+34.1% rel); Online-Mind2Web 37.0 vs 26.0 (+42.3% rel); Mind2Web-Live 25.0 vs 20.2 (+23.8% rel). 4-5× more efficient than tree search. Dreamer-7B specialist ? GPT-4o on online benchmarks.
Distinctive contribution	An early open demonstration that LLM-as-WM + 1-step MPC beats tree search on web tasks where irreversibility makes backtracking infeasible.

8.2 OS / computer-use agents

Reinforcement World Model Learning for LLM-based Agents (2602.05842) and World Models as an Intermediary between Agents and the Real World (2602.00785) generalize the lens: a learned WM mediates between LLM and expensive environment.

Deep dive — Dyna-Think (2506.00320)

What it models	A single Qwen2.5-32B that internalizes world-model simulation inside its `<think>` block — predicting next observation, action critique, or state-difference — for computer-use agents.

Training	DIT (imitation learning): few-shot prompt GPT-4o to reconstruct DeepSeek-R1's CoT keeping only WM-simulation-related text; SFT on cleaned trace. DDT (Dyna-style RL): online rollouts feed three WM objectives — next-state, state-diff, critic — jointly with rejection-sampled policy training.
Headline results	OSWorld BoN All 43.1 (DDT, 32B) vs R1-685B 44.8 — matches 685B model at 5% of parameters and 2× fewer tokens. WindowsAgentArena 34.9 vs Qwen2.5-32B 23.9 / R1 26.9. World-model accuracy correlates with task success at r=0.32 across models.
Distinctive contribution	Among the first Dyna-Q-style integrations where a single LLM hosts both policy and world model with critique-prediction as the WM objective.

8.3 SWE agents

SWE-bench (2310.06770) and SWE-Gym (2412.21139) defined the eval and training environment respectively. CodeAct (2402.01030) made the Python interpreter the unified action space. Reflexion (2303.11366) was the earliest entry with episodic verbal RL. Nanbeige SWE-World (2602.03419) trains a learned Docker-free execution surrogate. Understanding by Reconstruction (2603.11103) reverses the development process to harvest agentic pretraining traces. SWE-TRACE (2604.14820) provides process-level reward modeling over trajectories. Self-Play SWE-RL (2512.18552) introduces adversarial bug-injection/repair self-play. Bootstrapping Coding Agents — The Specification Is the Program (2603.17399) reframes the SWE task itself as a programmatic spec.

The §16 empirical synthesis separates three regimes: execution-grounded open-weight model training (CWM, SWE-RL), execution-grounded agents with learned simulators (Nanbeige SWE-World), and scaffold evolution around closed-model executors (Darwin GM, Huxley GM). Under best-of-k or verifier-reranked protocols, several systems report 60–68% on SWE-bench Verified, but those numbers are not directly comparable to pass@1 model-training results, and the scaffold-evolved systems are not "32B open-weight world-model-trained" — they run frontier closed models inside an evolved harness.

---

9. RL with Execution as the World Signal

The model-based-RL framing — world model is what the policy plans over — has produced a clean lineage.

9.1 Deep dive — RLEF (2410.02089)

What it models	Iterative code synthesis as a POMDP — actions are full code responses, observations are formatted public-test execution feedback, rewards come from held-out private tests.
Data	CodeContests train (13,328 problems, 669 discarded for missing tests). Initial policies Llama-3.0/3.1-Instruct 8B / 70B.
Objective	PPO with `R(s,a) = r(s,a) − β · log π(a□ |c□ )/ρ(a□ |c□ )`. r = +1 if all tests pass, −1 if any fail, −0.2 for malformed output. Turn limit 3. Geometric-mean response probability for KL bias correction.
Headline results	Llama-3.1-70B + RLEF: 37.5 / 40.1 valid/test pass@1 with budget 1@3 (vs 25.9 / 27.5 baseline). 54.5 / 54.5 at 10@100, surpassing AlphaCode 41B+clustering. Transfers to HumanEval+ (78.6 → 80.4) and MBPP+. Random-feedback ablation removes all gain.
Distinctive contribution	An early clean demonstration that standard PPO on multi-turn execution feedback beats both SFT and few-shot for code agents; isolates that the model learns to *use* feedback, not just sample more.

9.2 Deep dive — SWE-RL (2502.18449)

What it models	Bug-fix as single-shot search/replace edit conditioned on issue + full file context. *No execution at training time.*
Data	273k high-quality PR seeds extracted from a raw GitHub PR corpus. Trained on Llama-3.3-70B-Instruct.
Objective	GRPO with rule-based reward `R(o) = compare(patch_pred, patch_gt)` via `difflib.SequenceMatcher` (continuous 0..1). −1 for format violations. Continuous reward beats discrete in ablation (34.8 vs 29.0 oracle-repair).
Headline results	41.0% SWE-bench Verified pass@1 with Agentless Mini scaffold. OOD: HumanEval+ 76.2 → 79.9; CRUXEval-O 61.9 → 75.5; MATH 70.9 → 73.7 — *SFT degrades on these while RL improves.*

Distinctive contribution	Continuous-similarity reward on PR patches without execution induces emergent self-reflection, multi-approach exploration, and divide-and-conquer reasoning that transfers OOD. The "world model" is implicit in the patch-similarity reward.

9.3 Process Reward Models

ExecVerify (2603.11226), SWE-PRM (2509.02360), DataPRM (2604.24198), ThinkPRM (2504.16828) form a cluster where the WM is a learned *evaluator* of partial trajectories. As §17 develops critically, this is not the same object as a forward world model — PRMs are critics with execution grounding. They cannot roll out, cannot simulate counterfactuals. Survey hygiene argues for keeping the distinction.

---

10. Planning and Search with Code World Models

10.1 Deep dive — RAP (2305.14992)

What it models	Generic reasoning MDP — state = textual configuration, action = step proposed by the same LLM, transition obtained by re-prompting the LLM as world model.
Method	MCTS-UCT over the reasoning tree. Rewards: action likelihood, state confidence (majority voting), self-evaluation (LLM "Is this correct?" probability), task heuristics.
Headline results	Blocksworld 4-step: RAP@10 = 0.86 (LLaMA-33B) vs CoT-pass@10 = 0.07 vs GPT-4+CoT = 0.63. LLaMA-33B+RAP beats GPT-4+CoT by 33% relative on plan generation. GSM8K: 51.6 (RAP+aggr) vs CoT+SC 46.8.
Distinctive contribution	Earliest clean formulation that repurposes the LLM as both policy and transition model under MCTS. The template every later "LLM-as-WM" paper extends.

Tree of Thoughts (2305.10601), AlphaZero-like Tree Search for LLM Decoding (2309.17179), Tree Search for LM Agents (2407.01476), and Mastering Board Games by External/Internal Planning with LMs (2412.12119) develop the search frame; the last gives the clearest contemporary recipe for learned tree-search with LLM-as-WM, straightforwardly transferable to code.

10.2 Execution-conditioned generation

Execution Guided Line-by-Line Code Generation (2506.10948) uses classifier-free guidance to condition next-token prediction on candidate-runtime outcomes. Jupiter (2509.09245) formulates notebook state as MCTS nodes. REPL-Plan (2411.13826) reuses a REPL state pool across tasks. Substrate is well-developed for short-horizon code-gen; less so for long-horizon multi-file SWE.

---

11. JEPA, Dreamer, and the Latent-Action Gap

LeCun's Joint Embedding Predictive Architecture (I-JEPA, 2301.08243) predicts in embedding space rather than pixel space. The Dreamer family — Hafner et al.'s DreamerV1 (1912.01603), DreamerV2 (2010.02193), and DreamerV3 (2301.04104), built around the Recurrent State-Space Model — has near-zero direct application to code. Two papers occupy the gap.

11.1 Deep dive — LLM-JEPA (2509.14252)

What it models	A *joint embedding* between two views of the same knowledge — Text (NL prompt) and Code (e.g., SQL). Not a temporal world model; an embedding-space abstraction objective.
Architecture	Predictor is tied-weights: a single `[PRED]` token (with K predictor tokens) is appended and the LLM re-runs to produce `Pred(Enc(·))`. A custom block-causal attention mask lets both views go through a single forward pass.
Objective	`L = Σ  L_NTP(text) + λ · d(Pred(Enc(Text)), Enc(Code))`. d = cosine. Encoder reuses last-layer last-token hidden state.
Headline results	Llama-3.2-1B on NL-RX-SYNTH FT: 71.46% vs 57.29% NTP-FT (+14.2). Spider: ~50% vs ~47%. GSM8K: ~32% vs ~32%. Top-100 singular values of `Enc(Text) − Enc(Code)` collapse by orders of magnitude.
Distinctive contribution	To our knowledge the first JEPA-style embedding-space objective added to a generative LLM that preserves NTP loss while inducing low-rank Text?Code mapping. Critical question (§17): is this really JEPA in the LeCun sense, or a regularizer on LM training?

11.2 Deep dive — CoLA (2503.21383)

What it models	An MDP over text where the LLM is the transition model and actions are discrete *latent* tokens from a learned codebook, not vocabulary tokens.
Data	Llama-3.1-8B base, continued-pretrained on 200GB from SlimPajama / StarCoder / Proof-Pile-2 / WuDao; policy trained on 100GB subset.
Three modules	Inverse-Dynamics Model `f_inverse(x₁:t, x□ ₊₁) → a□` implemented as VQ-VAE-style encoder with codebook C. Language World Model inserts the chosen latent action into the LLM embedding stream and decodes the next token. Policy π(a□
Planning	Action-level MCTS over latent-action subtrees. MCTS-Q variant uses Double-DQN over (prompt, response, reward) tuples.
Headline results	Math-500: 42.4 (CoLA+RL) vs 38.2 baseline. MCTS-Q on Math-500: 68.2 vs 63.0 baseline MCTS-Q. +11% averaged on math reasoning; 64% win rate on alignment.
Distinctive contribution	Among the first to *replace* the 128k token-level action space of an LLM with a small learned latent-action codebook for RL — making the action space tractable for tree search.

11.3 The gap

Despite CWM and dozens of LLM-as-world-model papers, *no public Dreamer/RSSM-style latent-imagination world model has been trained for SWE agents.* CWM rolls out in token space. CoLA is the closest concrete instance. UniZero (2406.10667) generalizes MuZero with transformers but is rarely instantiated on code. Genie (2402.15391) gives the vision-side template. JEPA for RL (2504.16591) extends the energy-based objective to RL.

Whether the gap matters is a live question developed critically in §17. The vision-domain pressure that motivated Dreamer's RSSM design (pixel-space rollout cost) does not exist for code, where state is small and the simulator is available. The argument *for* latent imagination rests on inference-time speed and the action-space compression CoLA demonstrates, not on rollout cost per se.

---

12. Specialized Domains

Diffusion code models. DiffuCoder (2506.20639), Dream-Coder 7B (2509.01142). Iterative denoising naturally accommodates plan-then-refine generation.

Decompilation and cross-language. SK2Decompile (2509.22114), SALT4Decompile (2509.14646).

Translation as semantic-simulation task. EquiBench (2502.12466) supplies the equivalence eval.

Hardware / RTL. VeriRL (2508.18462), ChipSeek (2507.04736), VeriCoder (2504.15659) form a cluster where the simulator is the world model. Hardware is an attractive domain because simulators are precise, fast, and deterministic — closer to Atari than to Python.

ARC and abstract synthesis. Executable World Models for ARC-AGI-3 (2605.05138) instantiates literal-WM-per-task: synthesize a Python world model verified against observations. SOAR (2507.14172) evolves programs over ARC. Darwin / Huxley Godel Machines (2505.22954, 2510.21614) close the self-improvement loop.

---

13. Reasoning, Process Rewards, Memory

Long-CoT reasoning for code. o1-Coder (2412.00154) replicates o1 with MCTS+RL. R1-Code-Interpreter (2505.21668) supplies the open SFT+RL recipe across 144 tasks. Scaling Test-Time Compute to Achieve IOI Gold Medal (2510.14232) shows open-weight gpt-oss-120b matching closed reasoning models via inference-time scaling.

Long-CoT reasoning is *mental execution* — the chain-of-thought simulates the world model the network never explicitly trained. CWM-style explicit world modeling and R1-style reasoning are partial substitutes; whether they compose multiplicatively is open.

Memory. Episodic Memory is the Missing Piece for Long-Term LLM Agents (2502.06975) frames the gap. Memory as Action (2510.12635) treats memory operations as RL-learnable actions. RepoGraph (2410.14684) provides a durable repo-level dependency graph.

---

14. Verification, Probing, Safety

14.1 Formal verification: the leading edge

The verifier-grounded lineage is the only research direction in the corpus that does not rely on LLM self-report for correctness — the verifier provides ground truth.

Deep dive — ATLAS (2512.10173)

What it models	Verifier-grounded synthesis of Dafny programs (specification + implementation + proof annotations) from NL + Python reference + tests.
Data	TACO-verified yields 2,751 verified Dafny programs decomposed into 19,385 training examples across 6 tasks: NL-to-Code, NL-to-Spec, Spec-to-Code, Spec-Repair, Impl-Repair, Proof-Infilling. Base: Qwen-2.5-Coder-7B + LoRA.

Spec quality	Three lemma types: Soundness (contract holds on test inputs), Completeness-Contradiction (negated output ? false), Completeness-Perturbation (contract rejects structurally perturbed outputs).
Headline results	DafnyBench Pass@1: 32.4 ? 55.8 (+23.4); Pass@10 ? 56.9. DafnySynthesis Pass@5: 15.8 ? 65.8 (+50), surpassing GPT-4 (53.4) by 12.4 points at 7B.
Distinctive contribution	An early end-to-end pipeline for scaling verified-code data the way auto-formalization scaled Lean theorems. Operational soundness/completeness criteria via SMT-discharged lemmas filter degenerate specs.

The cluster also includes Re:Form (2507.16331, Dafny+RL), CLEVER (2505.13938, Lean), VeriStruct (2510.25015, Verus), AutoRocq (2511.17330, Rocq), and Semantic Equivalence Self-Play with Formal Verification (2604.17010, Liquid Haskell).

CLEVER's ?1/161 end-to-end Lean result is the most sobering number in the corpus: frontier models, with Lean type-checker access for self-verification, still fail on >99% of HumanEval-derived problems requiring joint spec + implementation verification. Understanding Formal Reasoning Failures in LLMs as Abstract Interpreters (2503.12686) is the diagnostic: when asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, all frontier reasoning models make systematic errors in widening, fixpoint termination, and join operations.

14.2 Symbolic execution and LLMs

AutoBug (2505.13452), SESpec (2506.09550), LLM-Sym (2409.09271), Loop Invariant Generation via Reasoning LLMs + SMT (2508.00419) combine LLMs with concrete or symbolic engines. The unifying pattern: the LLM hypothesizes the world model; symbolic execution verifies or extends it.

14.3 Probing and mechanistic interpretability

Mechanistic Interpretability of Code Correctness via SAEs (2510.02917) and On LLMs' Internal Representation of Code Correctness (2512.07404) ask what code LLMs actually represent. Findings: *partial, brittle* internal execution representations — a vindication of explicit trace pretraining.

14.4 Repair and debugging as world-model probing

Self-Debug (2304.05128), InspectCoder (2510.18327), Agent That Debugs — Dynamic State-Guided Vulnerability Repair (2504.07634), Agentic Code Reasoning (2603.01896, semi-formal execution-path reasoning without running code). Shared pattern: maintain a belief over program state, query the runtime to update the belief, act on the posterior — Bayesian world-modeling in everything but name.

14.5 Safety and malicious code

The Double Life of Code World Models (2512.13821) repurposes CWM-style trace predictions for malicious-behavior detection. CodeBreaker (2406.06822) is the offensive analogue. Concolic Execution + LLM for Zero-Day Malware Detection (2603.09044) pairs path-prioritization with concrete execution.

---

15. Benchmarks and the Evaluation Gap

Benchmarks split cleanly by what they measure.

- Static code quality — HumanEval, MBPP, LiveCodeBench (2403.07974) measure code-LLM output without exercising the world-model claim.
- Execution reasoning — CRUXEval (2401.03065), REval (2403.16437), CRUXEval-X (2408.13001), TraceEval (2605.11006), PLSemanticsBench (2510.03415). The canonical WM evals.
- Semantic equivalence — EquiBench (2502.12466), CodeARC (2503.23145).
- Agentic — SWE-bench, SWE-Gym, WebArena (2307.13854), Mind2Web (2306.06070), PyBench (2407.16732), OSWorld, WindowsAgentArena.

The eval gap. No widely adopted benchmark directly measures world-model fidelity by holding the policy fixed and varying the WM. Every measurement of WM capability is mediated through downstream task performance, so we cannot distinguish *the model has internalized program semantics* from *the model is exploiting trace-token shortcuts in the test distribution*. Demystifying Errors in LLM Reasoning Traces (2512.00215) supplies the diagnostic: even DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4, when asked to simulate execution and explain their reasoning, produce traces with errors clustering into nine categories (Computation, Indexing, Control Flow, Skip Statements, Misvaluation of Native API, Hallucination, Input Misread, etc.). Models with 85–98% final-answer accuracy on output prediction produce traces with systematic errors throughout. The decoupling — high outcome accuracy, low process fidelity — is the signature of a system that has learned to predict outcomes without faithfully simulating dynamics.

---

16. Empirical Landscape

16.1 SWE-bench scoreboard

Protocols mix in SWE-bench reporting and are non-comparable across the obvious axes. We split the table into three protocol classes and ask readers to compare within, not across, classes. Training / Evaluation regime: *Frozen-model scaffold* = no model training (agent loop around a frozen closed model); *EG-LLM SFT* = supervised fine-tune on agent trajectories; *EG-LLM + RL* = RL with execution rewards; *EG-agent + learned simulator* = SFT+RL plus a learned execution surrogate; *EG-LLM + trace mid-train + RL* = the CWM lineage; *Scaffold-evolved* = recursive scaffold/agent self-improvement around a frozen closed model. WM-Arch is reserved for systems with explicit transition/rollout machinery (N1/N2/N3 from §3); no row in this table qualifies. Tier is the evidence grading from §2 (A peer-reviewed, B major arxiv preprint with code, C single-author/institution, D blog/leaderboard).

Table 16.1a — Single-sample resolution (pass@1 on Verified unless noted)

System	Base model	Bench	Pass@1	Regime	Tier	Source
SWE-agent	GPT-4 Turbo	full	12.5%	Frozen-model scaffold	A	2405.15793
AutoCodeRover	GPT-4	Lite	19.0%	Frozen-model scaffold	A	2404.05427
Agentless	GPT-4o	Lite	32.0%	Frozen-model scaffold	B	2407.01489
SWE-Gym	Qwen-2.5-Coder-32B	Verified	20.6%	EG-LLM SFT	B	2412.21139
Agent-RLVR (no RM)	Qwen-2.5-72B	Verified	22.4%	EG-LLM + RL	B	2506.11425
Long-Context Multi-Turn RL	Qwen-2.5-72B	Verified	39.0%	EG-LLM + RL	B	2508.03501
SWE-RL (Llama3-SWE-RL-70B)	Llama-3.3-70B	Verified	41.0%	EG-LLM + RL	B	2502.18449
Nanbeige SWE-World (RL)	Qwen-2.5-Coder-32B	Verified	55.0%	EG-agent + learned simulator	B	2602.03419
CWM	CWM-32B	Verified	*not directly reported* †	EG-LLM + trace mid-train + RL	B	2510.02387

† The CWM paper does not report a pure pass@1 number under the standard protocol; the headline 65.8% in Table 16.1b is best@16 with verifier reranking. Inferred pass@1 from the paper's reported figures is approximately 53–55%, but this is *our estimate, not a directly reported number*, and we do not place it in the main row.

Table 16.1b — Best-of-k with verifier reranking / TTS

System	Base	Bench	Score	Sample budget	Tier	Source
Agent-RLVR + RM rerank	Qwen-2.5-72B	Verified	27.8%	rerank over multi-sample	B	2506.11425
Nanbeige SWE-World (TTS@8)	Qwen-2.5-Coder-32B	Verified	68.2%	best@8	B	2602.03419

System	Base	Bench	Score	Sample budget	Tier	Source
CWM (TTS)	CWM-32B	Verified	65.8%	best@16 over 40 reranked	B	2510.02387

Table 16.1c — Scaffold-evolution and self-play around closed-model executors

System	Backbone executor	Bench	Score (start → end)	Note	Tier	Source
Darwin Godel Machine	Claude-3.5-Sonnet	Verified	20% → 50%	Scaffold evolves over 80 iterations	B	2505.22954
Huxley Godel Machine	GPT-5-mini	Verified (500)	— → 61.4%	Scaffold optimized for Verified-60	B	2510.21614
SICA	Claude-3.5-Sonnet + o3-mini	Verified (subset)	17% → 53%	Scaffold evolves; not a model-training method	C	2504.15228

Citations that quote "CWM 65.8% SWE-bench Verified" without disclosing the best@16/TTS protocol should be read as misreporting, not as direct pass@1 numbers.

What can and cannot be compared. Within Table 16.1a the comparable axis is pass@1. SWE-RL's 41.0%, Long-Context-MT-RL's 39.0%, and Nanbeige SWE-World's 55.0% are roughly comparable as "open-weight WM-trained pass@1 on Verified." Table 16.1c sits in a different category — these are scaffold-evolution methods over closed models, and treating them as evidence for "world-model training works" conflates scaffold search with model improvement. Reading them inside Table 16.1a, as some online discussion does, is apples-to-oranges.

Two empirical claims, separately defensible. (i) On *pass@1*, open-weight WM-trained 32B systems (Nanbeige 55.0%, Long-Context-MT-RL 39.0%, SWE-RL 41.0%) outperform frozen open-weight baselines (Qwen-2.5-Coder-32B raw 6.2%) by 30–50 absolute points; this gain is the strongest case for training on agent trajectories with execution feedback. (ii) On *best@k with verifier reranking* (Table 16.1b), the same models reach 65–68%, but the additional 13–15 points are attributable to TTS reranking infrastructure, not to the WM training. Conflating (i) and (ii) by quoting CWM's 65.8% alongside SWE-RL's 41.0% as both "world-model-trained pass@1 SOTA" is exactly the misreporting the protocol split is designed to prevent.

16.2 Trace pretraining gains on execution-reasoning

Paper	Backbone	Baseline	After trace pretrain/FT	Delta	Benchmark	Tier
TRACED	UnixCoder	—	+12.4% rel branch-coverage; +25.2% rel variable-value	rel	CodeNet exec	A
NExT	PaLM 2-L	23.2	49.3	+26.1 abs	MBPP-R	A
NExT	PaLM 2-L	32.2	42.5	+10.3 abs	HumanEvalFix Plus	A
SemCoder (1.3B)	DS-Coder 1.3B	base	63.6 / 63.9	+23 abs	CRUXEval-I / O	B
“What I cannot execute”	Llama-3.1-8B	37.8%	~80%	+42 abs	CRUXEval-O	B
Do Code Semantics Help?	DSCoder, Llama-3, Gemma-2	various	□ a few abs; some regressions	mixed	comprehensive	B

For under-trained □8B open-weights, trace pretraining delivers +15 to +42 absolute on CRUXEval-O. The “Do Code Semantics Help?” ablation (2509.11686) is the disconfirming evidence: across multiple backbones and five trace representations, no single representation consistently outperforms others, and several downstream tasks regress under trace augmentation. The gain shrinks rapidly with base-model quality. Trace pretraining is a remedial intervention for weak code models; whether frontier models still benefit is unsettled.

16.3 Web/OS agents from WMs

Paper	Benchmark	Base	With WM	Delta	Mechanism	Tier
WebDreamer (GPT-4o WM)	VisualWebArena	17.6%	23.6%	+34.1% rel (□+6 abs)	LLM-as-WM + MPC	B
WebDreamer	Online-Mind2Web	26.0%	37.0%	+42.3% rel (□+11 abs)	same	B
Dreamer-7B (trained WM)	VisualWebArena	base	+4.7 abs	—	trained WM	B
WMA (2410.13232)	WebArena	base	action-selection 52□70%	—	trained transition WM	B
Dyna-Think DDT (32B)	OSWorld BoN	RFT~28%	43.1%	□+15 abs	Dyna-Q + WM head	B

Paper	Benchmark	Base	With WM	Delta	Mechanism	Tier
Dyna-Think DDT	WindowsAgentArena	28.4%	34.9%	+6.5 abs	same	B

WM gains on web/OS are real but quantitatively small (□+5–10 absolute task success rate on most benchmarks), and partly confounded with the extra synthetic data the WM generates. DyMo's 90%+ state-prediction accuracy versus 72.8% task success rate exemplifies the decoupling: WM heads can be accurate without the agent being accurate.

16.4 Formal verification vs LLM-only

System	Language	Benchmark	Baseline	With system	Tier	Source
ATLAS	Dafny	DafnyBench Pass@1	32.4%	55.8%	B	2512.10173
ATLAS	Dafny	DafnySynthesis Pass@5	15.8%	65.8% (>GPT-4 53.4)	B	2512.10173
CLEVER	Lean 4	161 problems end-to-end	best frontier	□1/161	B	2505.13938
VeriStruct	Verus / Rust	11 modules	—	99.2% (128/129 fns)	C	2510.25015
AutoRocq	Rocq	math + verif lemmas	5 baselines	48.0% math / 30.9% verif	B	2511.17330
Semantic Equiv Self-Play	Liquid Haskell	EquiBench	base	+13.3 pp	B	2604.17010

Verified codegen has the steepest training-data sensitivity in the survey: small synthetic datasets (2.7K verified Dafny programs in ATLAS) produce +25–50 absolute gains because LLM-only baselines start near zero. CLEVER's □1/161 shows that without explicit data/scaffolding, frontier models cannot reliably produce verified code. VeriStruct shows that on curated targets near-perfect is reachable.

16.5 Reasoning-model competitive programming

Model	Codeforces	IOI / ICPC	Tier
gpt-4o	808 (11th pct)	—	D
o1-preview	1258 (62nd pct)	—	D
o1	1673 (89th pct)	—	D
o1-ioi	1807 (~93rd pct)	IOI 2024 49th pct live	B

Model	Codeforces	IOI / ICPC	Tier
o3	elite-human-class	IOI 2024 gold	D
gpt-oss-120b + GenCluster	—	IOI 2025 gold (open-weight)	B
Gemini 2.5 Pro Exp	—	ICPC-Eval Pass@1 22.0%	B
DeepSeek-R1	—	ICPC-Eval Pass@1 14.4%	B
Claude 3.7 Sonnet	—	ICPC-Eval Pass@1 11.8%	B
GPT-4o	—	ICPC-Eval Pass@1 5.9%	B

Codeforces +999 rating points in 14 months on the o-series. The same line shows that frontier reasoning models gain almost everything from RL scaling, not from domain-specialized scaffolds — which complicates the case that "trace-style world models" are doing causal work for frontier systems.

---

17. Critical Perspectives

This section names where the field overclaims, where the consensus is fragile, and where vocabulary is doing more work than evidence. We develop seven theses.

17.1 The "world model" label has become marketing for any code LLM trained on something other than raw source

This is a terminology argument, not a contradiction with §3. §3 deliberately admits "implicit WMs in token policies" as a fourth flavor — including CWM, TRACED, SemCoder under that flavor — because that *is* how the field currently uses the term. The claim of §17.1 is that this permissive definition is what allows the rhetorical sleight of hand, and that a stricter definition would be more useful going forward.

Concretely: read CWM (2510.02387) carefully and the architecture it ships is a 32B decoder-only Transformer with GQA, sliding-window blocks, RoPE, AdamW — a Llama-class model. What earns it the "world model" badge is the mid-training datamix: 5T tokens of Python observation-action traces plus ForagerAgent SWE trajectories. There is no separate dynamics head, no inverse model, no recurrent latent. The same observation lands on LLM-JEPA, DyMo, and most "world model" papers from 2024–2026: the artifact is a standard LLM with an enriched objective.

A useful purity test: *can we ablate the supposed world-modeling component without changing the architecture?* If yes, the system is a trace-trained LLM. If no, there is a genuine architectural commitment. By that test, CoLA (2503.21383) and the Dreamer-for-LLMs gestures pass. CWM, SemCoder, NExT, and most of the "explicit WM" cluster fail. We propose that "world model" be reserved for systems with the architectural commitment, and that *execution-grounded code LLM* serve for the rest. The §3 permissive definition can stay as descriptive — what the field currently calls a code WM — but a normative tightening is warranted, and the survey from §17 onward uses the stricter sense.

17.2 Trace pretraining has a causal-isolation problem the surface numbers obscure

"Do Code Semantics Help?" (2509.11686) is the most damaging paper for the prevailing optimism. It runs a comprehensive ablation across DeepSeek-Coder, LLaMA-3, and Gemma-2 with five representations (Scratchpad, NExT, CodeExecutor, Concise, SemCoder) on program repair, code synthesis,

BigCodeBench, LiveCodeBench, and CRUXEval. Its headline: integrating trace-based semantic information into SFT *cannot significantly enhance* code-generation ability. In 7 of 9 synthesis settings the no-trace baseline wins or ties. At inference, in 36 of 56 test-scaling configurations, trace prompts hurt.

"What I cannot execute, I do not understand" (2503.05703) is gentler — Execution Tuning reaches ~80% CRUXEval-O — but the same paper's downstream evaluations on HumanEval, MBPP, and GSM8K show *negligible* gains from trace data in the SFT mix.

The strongest counterexample to "barely transfers" is NExT (2404.14662), which pushes PaLM 2-L on Mbpp-R from 23.2% to 40.8% *with traces removed at test time* — a +17.6 absolute gain on the no-trace-at-inference setting, which is exactly the transfer pattern the skeptical reading says shouldn't happen. The honest version of the thesis is therefore narrower: trace pretraining transfers to *program-repair* tasks where the model needs to reason about a buggy program's behavior (where NExT and TraceFixer-style work shines), and barely transfers to *fresh code synthesis* on benchmarks like HumanEval and MBPP that are closer to the model's prior. The "Do Code Semantics Help?" disconfirmation is on synthesis; NExT's transfer is on repair. Both can be true.

The honest reading: trace pretraining helps execution prediction (the thing trained on), transfers to runtime-reasoning-heavy downstream tasks like repair, and barely transfers to fresh code synthesis, exactly the pattern you would expect if dense execution supervision is teaching a runtime-tracking skill rather than a generative model of program intent.

CWM's 65.8% on SWE-bench is the apparent counterexample, but the Meta team reports it after trace mid-training *and* 3M ForagerAgent SWE trajectories *and* multi-task RL with verifiable rewards — three interventions stacked. The "world model" component is not causally isolated from the SWE-trajectory and RL components. Without an ablation that removes trace mid-training while holding ForagerAgent and RL fixed, the headline is unfalsifiable. The field has agreed to call CWM a world-model success because the name is on the model card, not because the experimental design demonstrates it.

17.3 The Dreamer-for-code gap may be a non-problem

The conventional framing treats the absence of latent-imagination world models for SWE as the field's largest architectural gap. The empirical record argues the opposite. CWM in token space reaches 65.8% SWE-bench. CoLA produces respectable but not field-shifting results, and even there the WM is fine-tuned on top of a standard LLM rather than replacing it.

Vision world models needed latent rollouts because pixel-space rollouts were too expensive — a frame is ~10^6 dimensions, dynamics are partially observed. Program execution is the opposite: a Python frame is small, dynamics are observable, and the simulator (CPython) is available for free at training time. The pressure that drove Dreamer's RSSM design does not exist for code.

Debugging Code World Models (2602.07672) shows CWM's long-horizon failures are dominated by *action hallucination*, not state-propagation error — under teacher forcing CWM tracks state correctly for 128 steps. A latent rollout would compress states but not fix the action policy, which is the actual bottleneck.

The counterargument is fair: latent rollouts permit faster planning at inference, and for multi-agent or population-scale search the speedup is asymptotically meaningful. But the survey should retire "single largest architectural gap" framing and replace it with "an interesting open question whose payoff is not yet demonstrated."

17.4 PRMs are critics, not world models

Process reward models — ExecVerify (2603.11226), SWE-PRM (2509.02360), ThinkPRM (2504.16828), FunPRM (2601.22249), DataPRM (2604.24198) — are often grouped under the world-modeling umbrella. This is wrong in a way that matters. A world model is, by every definition the survey uses, a *forward* predictor of `(state, action) → next_state`. A PRM is a *backward-looking evaluator*: given a partial trajectory, score it. In classical model-based RL these are different objects — Dreamer has both a world model (RSSM) and a critic (value function).

PRMs cannot roll out. Cannot simulate counterfactuals. Cannot be used by a planner that wants to score a hypothesized future. Conflating them dilutes the world-model concept until it means "any neural network trained on execution-related signals" — at which point the term is useless. Vocabulary discipline is cheap and the field would benefit from it.

The same critique applies, less severely, to verifier-grounded systems: a Lean proof checker is not a world model, it is a deterministic verifier of a candidate output. Calling it "the world model" when ATLAS, Re:Form, or AutoRocq use it makes for a tidy survey arc but blurs the actual computational structure.

17.5 "General Agents Contain World Models" is much weaker than its title suggests

Richens et al. (2506.01622) prove that any goal-conditioned policy satisfying a regret bound δ for sufficiently deep composite goals (depth $n \square 1$) must encode an extractable approximation of the transition function with bounded error. Genuine and elegant.

But read the assumptions: fully observed environment, finite communicating stationary controlled MDP, goal-conditioned policy satisfying a regret bound for a specific class of LTL composite goals of depth n. Theorem 2 of the same paper explicitly shows that for myopic agents (depth-1 goals), *no world model is needed*. Real SWE agents are myopic-ish over short turns and approximately competent over longer ones; their environments are partially observed (rarely full filesystem state); they violate stationarity (the repository changes under their actions); and their regret bound for arbitrary composite goals is unknown and almost certainly not satisfied. The authors caveat this in §6 ("Limitations") of their paper.

The theorem is a beautiful existence proof for an idealized agent class. It is *not* an empirical statement that SWE coding agents have learned world models, and it provides no guidance about the fidelity of any world model they may have learned.

17.6 The verifier-grounded lineage is the actual leading edge

ATLAS, Re:Form, CLEVER, VeriStruct, AutoRocq, and the Liquid Haskell self-play paper (2604.17010) share a property no LLM-only system possesses: code whose correctness is *machine-checked* against a formal specification. Compare to the SWE-bench paradigm, where "correctness" means "hidden unit tests pass" — a weaker guarantee, since unit tests cover specific inputs and the system can pass them while being wrong on adjacent inputs.

The abstract-interpreter paper (2503.12686) is the diagnostic: when frontier reasoning LLMs are asked to reason in the style of formal abstract interpretation over 22 SV-COMP programs, they make systematic errors in widening, fixpoint termination, control-flow propagation, and meet/join operations. They generated unsound invariants on programs as small as `count_by_2.c`. If LLMs cannot reliably perform

interval-domain abstract interpretation on toy C programs, claims that they have learned faithful internal world models of program semantics are doing a lot of inferential work.

The verifier-grounded line is the only research direction that does not rely on LLM self-report for correctness. Scoped carefully, it leads on *synthesis-from-spec* (ATLAS reaches 65.8% Pass@5 on DafnySynthesis at 7B, surpassing GPT-4) and on *near-perfect verified completion of curated targets* (VeriStruct 99.2% on 11 Verus modules). It does *not* lead on end-to-end verified codegen from natural language: CLEVER reports ?1/161 on Lean problems requiring joint spec + implementation verification. The future of *correct* code is plausibly hybrid — neural proposal, symbolic verification, with the verifier providing ground truth that learned world models cannot — but the hybrid is not yet a finished pillar. We catalog the verifier line in §14.1 alongside symbolic execution and abstract-interpretation probing; readers who accept this thesis should weight that subsection more heavily than its sequence position suggests.

17.7 The evaluation gap is the structural reason the field looks confused

Across CRUXEval, REval, CRUXEval-X, PLSemanticsBench, TraceEval, and EquiBench, no benchmark holds policy fixed and varies world-model quality. Every measurement of "world-modeling capability" is mediated through downstream task performance, so we cannot distinguish *the model has internalized program semantics* from *the model is exploiting trace-token shortcuts in the test distribution*.

"Demystifying Errors in LLM Reasoning Traces" (2512.00215) is the diagnostic: even DeepSeek-R1, o4-mini, Gemini 2.5 Flash, and Claude 4, when asked to simulate execution, produce traces with errors in nine systematic categories. Models with 85–98% final-answer accuracy on output prediction produce traces with systematic errors throughout — high outcome accuracy, low process fidelity, the signature of a system that predicts outcomes without faithfully simulating dynamics.

Self-repair literature exhibits the same pathology: Olausson et al. (2306.09896) showed that GPT-4 self-repair on APPS and HumanEval, normalized by compute, often performs *worse* than i.i.d. resampling. The bottleneck is the model's feedback quality, not its repair capability — human-written feedback boosts repair success by 1.58×. The model can generate code, can sometimes recognize bugs, but cannot reliably simulate why its code is wrong — which is exactly what a faithful world model would let it do. The empirical bound on LLM self-repair is, in effect, an empirical bound on the fidelity of the implicit world model the LLM is running. Calling that world model internal is fine; calling it good is not.

Until benchmarks measure process fidelity independently of outcome, "is this system actually building a world model?" remains scientifically undecidable.

---

18. Open Problems

The critical perspectives of §17 reshape the conventional open-problems list. We propose six problems where the literature is thinnest *and* the upside is largest.

1. Causal isolation of trace-pretraining contributions. Every claim of the form "this WM-trained model achieves X" should be paired with an ablation removing the WM component while holding training data and RL fixed. CWM in particular needs this. Without it, the headline numbers underdetermine whether the WM did the work.

2. World-model fidelity as a first-class metric. §15's eval gap is concrete: build a benchmark where holding policy fixed and varying WM quality causes measurable variation in planning quality, independent of downstream task. This benchmark would clarify the field more than any single new model.

3. Hybrid neural-symbolic systems. §17.6 argues the verifier-grounded line is the leading edge. The natural integration is *neural proposal, symbolic verification*, with the verifier providing gradient-free correctness signal and the neural component providing proposals at scale. Differentiable surrogates of symbolic verifiers (Lean / Dafny / Rocq) that pass verifier-style gradients during training are open.

4. Multi-modal WMs for coding. GUI agents need pixel-level WMs (Neural Computers, 2604.06425, is a first attempt). Tying pixel WMs to code-state WMs through a shared latent is essentially unsolved.

5. Long-horizon credit assignment with execution-grounded rewards. PRMs (§9.3) are early, and §17.4 argues they should be conceptually separated from world models. The right structure for rewarding an agent across hundreds of execution-grounded steps is a live question.

6. World models of the developer, not just the program. All current WMs model the *machine*. Few model the *developer intent* with comparable fidelity. ATLAS and Re:Form gesture in this direction by treating the spec as the WM. A full developer-intent WM would close the agentic loop.

We do not list "Dreamer-for-SWE-agents" as the field's largest gap, contrary to common framing. §17.3 argues the pressure motivating that direction in vision does not transfer to code. It remains an interesting research question, not the highest-leverage one.

Three under-explored representations. The §5.3 taxonomy table flags three classes of representation, mature in vision-WM, that have no code-WM exemplar:

- *Global Latent Vector (Dreamer-style RSSM)* — discussed and contested above. The Hafner et al. DreamerV1–V3 line proves the design space exists; whether it pays off for code is the question §17.3 leaves open.
- *Spatial / Structural Grid* — the analog of OccWorld / BEV for code would be a learned predictive grid over AST nodes, call-graph edges, or CFG states. RepoGraph (2410.14684) shows the static version is useful as agent state; the predictive version is unexplored.
- *Decomposed Object / Slot (object-centric WMs)* — the analog for code would model variables, scopes, or classes as discrete persistent slots whose state propagates independently. No paper in the corpus instantiates this, despite obvious mappings (each variable is an object, each frame is a scene).

The object-centric and structural-grid gaps look more genuine than the Dreamer one, in our reading, because they exploit structure that code *already has* (objects = variables, grids = AST/CFG) rather than borrowing pressure from a domain (vision) where the structural assumptions differ.

---

19. Conclusion

Across the literature surveyed here, a single trajectory is visible: from neural execution (modeling the machine), through trace pretraining (modeling execution implicitly), to CWM and its descendants (modeling execution explicitly with a named artifact), to agentic SWE and RL (modeling the environment), to JEPA and latent-action models (modeling in compressed space), and on toward formal verification, probing, and safety (modeling reliably). What was a scattered set of insights in 2014 has by 2026 cohered into a recognizable research program with a recognizable artifact — the code world model.

The remaining work splits into two halves. The first is empirical: close the eval gap, isolate the causal contribution of WM-training, build hybrid neural-symbolic systems whose correctness is verifier-checkable rather than test-checkable. The second is rhetorical: hold the term "world model"

to a strict definition so the literature can distinguish architectural commitments from training-data choices, and resist the temptation to oversell extractability theorems and latent-imagination analogies whose premises do not transfer to code.

The opportunity is large precisely because the framework is now clear enough to identify what is missing. The work to do is the work this survey has tried to make visible.

Appendix A · References

All citations in this survey use arxiv identifiers inline (e.g. `2510.02387`). Each cited identifier resolves to one entry below. For machine-readable access to the full 184-paper corpus (including non-cited entries considered during the survey passes), see `papers.json`.

- arxiv:1410.4615 — Learning to Execute. https://arxiv.org/abs/1410.4615
- arxiv:1511.06279 — Neural Programmer-Interpreters. https://arxiv.org/abs/1511.06279
- arxiv:1711.07163 — Dynamic Neural Program Embedding for Program Repair. https://arxiv.org/abs/1711.07163
- arxiv:1803.10122 — World Models. https://arxiv.org/abs/1803.10122
- arxiv:1906.07181 — Learning Execution through Neural Code Fusion. https://arxiv.org/abs/1906.07181
- arxiv:1912.01603 — Dream to Control - Learning Behaviors by Latent Imagination (DreamerV1). https://arxiv.org/abs/1912.01603
- arxiv:2010.02193 — Mastering Atari with Discrete World Models (DreamerV2). https://arxiv.org/abs/2010.02193
- arxiv:2010.12621 — Learning to Execute Programs with Instruction Pointer Attention Graph Neural Networks. https://arxiv.org/abs/2010.12621
- arxiv:2107.03374 — Evaluating Large Language Models Trained on Code. https://arxiv.org/abs/2107.03374
- arxiv:2108.07732 — Program Synthesis with Large Language Models. https://arxiv.org/abs/2108.07732
- arxiv:2112.00114 — Show Your Work - Scratchpads for Intermediate Computation with Language Models. https://arxiv.org/abs/2112.00114
- arxiv:2207.01780 — CodeRL - Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. https://arxiv.org/abs/2207.01780
- arxiv:2301.04104 — Mastering Diverse Domains through World Models (DreamerV3). https://arxiv.org/abs/2301.04104
- arxiv:2301.08243 — I-JEPA - Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. https://arxiv.org/abs/2301.08243
- arxiv:2302.08468 — LEVER - Learning to Verify Language-to-Code Generation with Execution. https://arxiv.org/abs/2302.08468
- arxiv:2303.11366 — Reflexion - Language Agents with Verbal Reinforcement Learning. https://arxiv.org/abs/2303.11366
- arxiv:2304.05128 — Teaching Large Language Models to Self-Debug. https://arxiv.org/abs/2304.05128
- arxiv:2305.05383 — Code Execution with Pre-trained Language Models. https://arxiv.org/abs/2305.05383
- arxiv:2305.10601 — Tree of Thoughts - Deliberate Problem Solving with Large Language Models. https://arxiv.org/abs/2305.10601
- arxiv:2305.14992 — Reasoning with Language Model is Planning with World Model - RAP. https://arxiv.org/abs/2305.14992
- arxiv:2306.06070 — Mind2Web - Towards a Generalist Agent for the Web. https://arxiv.org/abs/2306.06070
- arxiv:2306.07487 — TRACED - Execution-aware Pre-training for Source Code. https://arxiv.org/abs/2306.07487
- arxiv:2306.09896 — Is Self-Repair a Silver Bullet for Code Generation. https://arxiv.org/abs/2306.09896

- arxiv:2307.13854 — WebArena - A Realistic Web Environment for Building Autonomous Agents. https://arxiv.org/abs/2307.13854
- arxiv:2309.17179 — AlphaZero-like Tree-Search can Guide Large Language Model Decoding and Training. https://arxiv.org/abs/2309.17179
- arxiv:2310.06770 — SWE-bench - Can Language Models Resolve Real-World GitHub Issues. https://arxiv.org/abs/2310.06770
- arxiv:2401.03065 — CRUXEval - A Benchmark for Code Reasoning, Understanding and Execution. https://arxiv.org/abs/2401.03065
- arxiv:2402.01030 — Executable Code Actions Elicit Better LLM Agents. https://arxiv.org/abs/2402.01030
- arxiv:2402.15391 — Genie - Generative Interactive Environments. https://arxiv.org/abs/2402.15391
- arxiv:2403.07974 — LiveCodeBench - Holistic and Contamination Free Evaluation of Large Language Models for Code. https://arxiv.org/abs/2403.07974
- arxiv:2403.16437 — Evaluating Large Language Models with Runtime Behavior of Program Execution. https://arxiv.org/abs/2403.16437
- arxiv:2404.05427 — AutoCodeRover - Autonomous Program Improvement. https://arxiv.org/abs/2404.05427
- arxiv:2404.14662 — NExT - Teaching Large Language Models to Reason about Code Execution. https://arxiv.org/abs/2404.14662
- arxiv:2405.15383 — Generating Code World Models with Large Language Models Guided by Monte Carlo Tree Search. https://arxiv.org/abs/2405.15383
- arxiv:2405.15793 — SWE-agent - Agent-Computer Interfaces Enable Automated Software Engineering. https://arxiv.org/abs/2405.15793
- arxiv:2406.00515 — A Survey on Large Language Models for Code Generation. https://arxiv.org/abs/2406.00515
- arxiv:2406.01006 — SemCoder - Training Code Language Models with Comprehensive Semantics Reasoning. https://arxiv.org/abs/2406.01006
- arxiv:2406.06822 — An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models. https://arxiv.org/abs/2406.06822
- arxiv:2406.10667 — UniZero - Generalized and Efficient Planning with Scalable Latent World Models. https://arxiv.org/abs/2406.10667
- arxiv:2407.01476 — Tree Search for Language Model Agents. https://arxiv.org/abs/2407.01476
- arxiv:2407.01489 — Agentless - Demystifying LLM-based Software Engineering Agents. https://arxiv.org/abs/2407.01489
- arxiv:2407.16732 — PyBench - Evaluating LLM Agent on Various Real-World Coding Tasks. https://arxiv.org/abs/2407.16732
- arxiv:2408.13001 — CRUXEval-X - A Benchmark for Multilingual Code Reasoning Understanding and Execution. https://arxiv.org/abs/2408.13001
- arxiv:2409.09271 — Python Symbolic Execution with LLM-powered Code Generation. https://arxiv.org/abs/2409.09271
- arxiv:2410.02089 — RLEF - Grounding Code LLMs in Execution Feedback with Reinforcement Learning. https://arxiv.org/abs/2410.02089
- arxiv:2410.13232 — Web Agents with World Models - Learning and Leveraging Environment Dynamics in Web Navigation. https://arxiv.org/abs/2410.13232
- arxiv:2410.14684 — RepoGraph - Enhancing AI Software Engineering with Repository-level Code Graph. https://arxiv.org/abs/2410.14684
- arxiv:2411.06559 — Is Your LLM Secretly a World Model of the Internet - WebDreamer. https://arxiv.org/abs/2411.06559
- arxiv:2411.13826 — REPL-Plan - Interactive and Expressive Code-Augmented Planning with Large Language Models. https://arxiv.org/abs/2411.13826
- arxiv:2411.14499 — Understanding World or Predicting Future - A Comprehensive Survey of

World Models. https://arxiv.org/abs/2411.14499
- arxiv:2412.00154 — o1-Coder - an o1 Replication for Coding. https://arxiv.org/abs/2412.00154
- arxiv:2412.12119 — Mastering Board Games by External and Internal Planning with Language Models. https://arxiv.org/abs/2412.12119
- arxiv:2412.21139 — Training Software Engineering Agents and Verifiers with SWE-Gym. https://arxiv.org/abs/2412.21139
- arxiv:2501.12948 — DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. https://arxiv.org/abs/2501.12948
- arxiv:2502.06975 — Position - Episodic Memory is the Missing Piece for Long-Term LLM Agents. https://arxiv.org/abs/2502.06975
- arxiv:2502.12466 — EquiBench - Benchmarking LLMs' Reasoning about Program Semantics via Equivalence Checking. https://arxiv.org/abs/2502.12466
- arxiv:2502.18449 — SWE-RL - Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. https://arxiv.org/abs/2502.18449
- arxiv:2503.05703 — What I cannot execute, I do not understand - Training and Evaluating LLMs on Program Execution Traces. https://arxiv.org/abs/2503.05703
- arxiv:2503.12686 — Understanding Formal Reasoning Failures in LLMs as Abstract Interpreters. https://arxiv.org/abs/2503.12686
- arxiv:2503.21383 — CoLA - Controlling Large Language Models with Latent Actions. https://arxiv.org/abs/2503.21383
- arxiv:2503.23145 — CodeARC - Benchmarking Reasoning Capabilities of LLM Agents for Inductive Program Synthesis. https://arxiv.org/abs/2503.23145
- arxiv:2504.07634 — Agent That Debugs - Dynamic State-Guided Vulnerability Repair. https://arxiv.org/abs/2504.07634
- arxiv:2504.15228 — A Self-Improving Coding Agent. https://arxiv.org/abs/2504.15228
- arxiv:2504.15659 — VeriCoder - Enhancing LLM-Based RTL Code Generation through Functional Correctness Validation. https://arxiv.org/abs/2504.15659
- arxiv:2504.16591 — JEPA for RL - Investigating Joint-Embedding Predictive Architectures for Reinforcement Learning. https://arxiv.org/abs/2504.16591
- arxiv:2504.16828 — Process Reward Models That Think. https://arxiv.org/abs/2504.16828
- arxiv:2505.13452 — Large Language Model Powered Symbolic Execution. https://arxiv.org/abs/2505.13452
- arxiv:2505.13938 — CLEVER - A Curated Benchmark for Formally Verified Code Generation. https://arxiv.org/abs/2505.13938
- arxiv:2505.21668 — R1-Code-Interpreter - LLMs Reason with Code via Supervised and Multi-stage Reinforcement Learning. https://arxiv.org/abs/2505.21668
- arxiv:2505.22954 — Darwin Godel Machine - Open-Ended Evolution of Self-Improving Agents. https://arxiv.org/abs/2505.22954
- arxiv:2506.00320 — Dyna-Think - Synergizing Reasoning, Acting, and World Model Simulation in AI Agents. https://arxiv.org/abs/2506.00320
- arxiv:2506.01622 — General Agents Contain World Models. https://arxiv.org/abs/2506.01622
- arxiv:2506.02918 — World Modeling Improves Language Model Agents. https://arxiv.org/abs/2506.02918
- arxiv:2506.09550 — Integrating Symbolic Execution with LLMs for Automated Generation of Program Specifications. https://arxiv.org/abs/2506.09550
- arxiv:2506.10343 — Code Execution as Grounded Supervision for LLM Reasoning. https://arxiv.org/abs/2506.103
- arxiv:2506.10948 — Execution Guided Line-by-Line Code Generation. https://arxiv.org/abs/2506.10948
- arxiv:2506.11425 — Agent-RLVR - Training Software Engineering Agents via Guidance and Environment Rewards. https://arxiv.org/abs/2506.11425
- arxiv:2506.20639 — DiffuCoder - Understanding and Improving Masked Diffusion Models for

Code Generation. https://arxiv.org/abs/2506.20639
- arxiv:2507.04736 — ChipSeek - Optimizing Verilog Generation via EDA-Integrated Reinforcement Learning. https://arxiv.org/abs/2507.04736
- arxiv:2507.14172 — SOAR - Self-Improving Language Models for Evolutionary Program Synthesis on ARC-AGI. https://arxiv.org/abs/2507.14172
- arxiv:2507.16331 — Re-Form - Reducing Human Priors in Scalable Formal Software Verification with RL in LLMs on Dafny. https://arxiv.org/abs/2507.16331
- arxiv:2508.00419 — Loop Invariant Generation - A Hybrid Framework of Reasoning Optimised LLMs and SMT Solvers. https://arxiv.org/abs/2508.00419
- arxiv:2508.03501 — Training Long-Context, Multi-Turn Software Engineering Agents with Reinforcement Learning. https://arxiv.org/abs/2508.03501
- arxiv:2508.18462 — VeriRL - Boosting LLM-based Verilog Code Generation via Reinforcement Learning. https://arxiv.org/abs/2508.18462
- arxiv:2509.01142 — Dream-Coder 7B - An Open Diffusion Language Model for Code. https://arxiv.org/abs/2509.01142
- arxiv:2509.02360 — Act Like You're Paying for This - Course-Correcting Code Agents with PRMs. https://arxiv.org/abs/2509.02360
- arxiv:2509.09245 — Jupiter - Enhancing LLM Data Analysis Capabilities via Notebook and Inference-Time Value-Guided Search. https://arxiv.org/abs/2509.09245
- arxiv:2509.11686 — Do Code Semantics Help - A Comprehensive Study on Execution Trace-Based Information for Code LLMs. https://arxiv.org/abs/2509.11686
- arxiv:2509.14252 — LLM-JEPA - Large Language Models Meet Joint Embedding Predictive Architectures. https://arxiv.org/abs/2509.14252
- arxiv:2509.14646 — SALT4Decompile - Inferring Source-level Abstract Logic Tree for LLM-Based Binary Decompilation. https://arxiv.org/abs/2509.14646
- arxiv:2509.22114 — SK2Decompile - LLM-Based Two-Phase Binary Decompilation. https://arxiv.org/abs/2509.22
- arxiv:2510.02387 — CWM - An Open-Weights LLM for Research on Code Generation with World Models. https://arxiv.org/abs/2510.02387
- arxiv:2510.02917 — Mechanistic Interpretability of Code Correctness in LLMs via Sparse Autoencoders. https://arxiv.org/abs/2510.02917
- arxiv:2510.03415 — PLSemanticsBench - LLMs as Programming Language Interpreters. https://arxiv.org/abs/2510.03415
- arxiv:2510.12635 — Memory as Action - Autonomous Context Curation for Long-Horizon Agentic Tasks. https://arxiv.org/abs/2510.12635
- arxiv:2510.14232 — Scaling Test-Time Compute to Achieve IOI Gold Medal with Open-Weight Models. https://arxiv.org/abs/2510.14232
- arxiv:2510.16732 — A Comprehensive Survey on World Models for Embodied AI. https://arxiv.org/abs/2510.1673
- arxiv:2510.18327 — InspectCoder - Dynamic Analysis-Enabled Self Repair through Interactive LLM-Debugger Collaboration. https://arxiv.org/abs/2510.18327
- arxiv:2510.21614 — Huxley-Godel Machine - Human-Level Coding Agent Development. https://arxiv.org/abs/2510.21614
- arxiv:2510.25015 — VeriStruct - AI-assisted Automated Verification of Data-Structure Modules in Verus. https://arxiv.org/abs/2510.25015
- arxiv:2511.17330 — Agentic Program Verification. https://arxiv.org/abs/2511.17330
- arxiv:2512.00215 — Demystifying Errors in LLM Reasoning Traces - An Empirical Study of Code Execution Simulation. https://arxiv.org/abs/2512.00215
- arxiv:2512.07404 — On LLMs' Internal Representation of Code Correctness. https://arxiv.org/abs/2512.07404
- arxiv:2512.10173 — ATLAS - Automated Toolkit for Large-Scale Verified Code Synthesis.

https://arxiv.org/abs/2512.10173

- arxiv:2512.13821 — The Double Life of Code World Models - Provably Unmasking Malicious Behavior Through Execution Traces. https://arxiv.org/abs/2512.13821
- arxiv:2512.18552 — Toward Training Superintelligent Software Agents through Self-Play SWE-RL. https://arxiv.org/abs/2512.18552
- arxiv:2601.22249 — FunPRM - Function-as-Step Process Reward Model with Meta Reward Correction for Code Generation. https://arxiv.org/abs/2601.22249
- arxiv:2602.00785 — World Models as an Intermediary between Agents and the Real World. https://arxiv.org/abs/2602.00785
- arxiv:2602.03419 — Nanbeige SWE-World - Building Software Engineering Agents in Docker-Free Environments. https://arxiv.org/abs/2602.03419
- arxiv:2602.05842 — Reinforcement World Model Learning for LLM-based Agents. https://arxiv.org/abs/2602.058
- arxiv:2602.07672 — Debugging Code World Models. https://arxiv.org/abs/2602.07672
- arxiv:2603.01896 — Agentic Code Reasoning. https://arxiv.org/abs/2603.01896
- arxiv:2603.09044 — Synergistic Directed Execution and LLM-Driven Analysis for Zero-Day AI-Generated Malware Detection. https://arxiv.org/abs/2603.09044
- arxiv:2603.09951 — Towards a Neural Debugger for Python. https://arxiv.org/abs/2603.09951
- arxiv:2603.11103 — Understanding by Reconstruction - Reversing the Software Development Process for LLM Pretraining. https://arxiv.org/abs/2603.11103
- arxiv:2603.11226 — ExecVerify - White-Box RL with Verifiable Stepwise Rewards for Code Execution Reasoning. https://arxiv.org/abs/2603.11226
- arxiv:2603.17399 — Bootstrapping Coding Agents - The Specification Is the Program. https://arxiv.org/abs/2603.17399
- arxiv:2604.03144 — InCoder-32B-Thinking - Industrial Code World Model for Thinking. https://arxiv.org/abs/2604.03144
- arxiv:2604.03253 — Self-Execution Simulation. https://arxiv.org/abs/2604.03253
- arxiv:2604.06425 — Neural Computers. https://arxiv.org/abs/2604.06425
- arxiv:2604.14820 — SWE-TRACE - Optimizing Long-Horizon SWE Agents Through Rubric Process Reward Models and Heuristic Test-Time Scaling. https://arxiv.org/abs/2604.14820
- arxiv:2604.17010 — Improving LLM Code Reasoning via Semantic Equivalence Self-Play with Formal Verification. https://arxiv.org/abs/2604.17010
- arxiv:2604.20926 — Learning Reasoning World Models for Parallel Code. https://arxiv.org/abs/2604.20926
- arxiv:2604.24198 — DataPRM - Process-Level Reward Modeling for Agentic Data Analysis. https://arxiv.org/abs/2604.24198
- arxiv:2605.05138 — Executable World Models for ARC-AGI-3 in the Era of Coding Agents. https://arxiv.org/abs/2605.05138
- arxiv:2605.11006 — TraceEval - An Execution-Verified Multi-Language Benchmark for Code Semantic Reasoning. https://arxiv.org/abs/2605.11006

Appendix B · Glossary

- CWM — Code World Model (2510.02387 and lineage).
- JEPA — Joint Embedding Predictive Architecture (LeCun et al.).
- RSSM — Recurrent State-Space Model (Dreamer family).
- PRM — Process Reward Model.
- SWE agent — Software-engineering agent operating on real repositories.

- Trace pretraining — Pretraining where execution traces appear in input or target.
- Execution-grounded RL — RL whose reward derives from program execution.
- Latent-imagination rollout — Forward simulation in compressed latent space rather than token space.
- TTS — Test-time scaling (sampling many candidates + verifier reranking at inference).
- GRPO — Group Relative Policy Optimization (R1-family RL algorithm).